

PROJEKT BAZY DANYCH

System zarządzania apteką

PROJEKT GRUPOWY

zrealizowany w ramach zajęć na Politechnice Wrocławskiej
Wydział Informatyki i Telekomunikacji

Autorzy:

Robert Tworek

Kacper Wajda

Michał Gładkojć



Politechnika Wroclawska

1.1. Cel projektu

Celem projektu jest zaprojektowanie i wykonanie lokalnego systemu bazodanowego wspierającego codzienną pracę apteki w małym mieście. System integruje w jednej bazie danych:

- publiczną bazę recept (do odczytu i realizacji e-recept oraz recept papierowych),
- bazę leków gotowych dostępnych w aptece,
- bazę składników i receptur leków robionych na miejscu (receptura apteczna),
- bazę zamówień do hurtowni wraz z modułem automatycznego uzupełniania zapasów.

System jest realizowany jako aplikacja desktopowa w języku C# z interfejsem użytkownika opartym o bibliotekę Avalonia UI, korzystająca z relacyjnej bazy danych PostgreSQL.

1.2. Zakres projektu

Projekt obejmuje:

- zaprojektowanie i implementację relacyjnej bazy danych dla apteki,
- zaprojektowanie modelu logicznego i fizycznego bazy (tabele, relacje, klucze, ograniczenia),
- integrację danych dotyczących: recept, leków gotowych, surowców recepturowych, magazynu, zamówień, klientów, użytkowników i operacji,
- przygotowanie aplikacji dostępowej (klienckiej) umożliwiającej obsługę wszystkich kluczowych procesów apteki (praca offline, połączenie wyłącznie z lokalną bazą PostgreSQL),
- przygotowanie dokumentacji projektowej zgodnej z wymaganiami przedmiotu „Bazy danych – projekt”.

System jest przeznaczony do pracy w pojedynczej aptece (jedna lokalizacja), obsługiwanej przez 1–2 użytkowników jednocześnie.

- Leki gotowe – dane o produktach, producentach, partiach, datach ważności i ilościach.
- Surowce recepturowe – informacje o substancjach czynnych, pomocniczych i ich zapasach.
- Rejestr realizowanych recept – dane o wydanych lekach gotowych i recepturowych.
- Rejestr klientów apteki – dane o klientach zgodnie z RODO.
- Magazyn – obsługa dostaw, korekt, rezerwacji, stanów minimalnych i zużycia surowców.
- Zamówienia do hurtowni – automatyczne i ręczne, dla leków gotowych i surowców recepturowych.
- Raporty i rozliczenia – sprzedaż, zapasy, zużycie składników recepturowych, rozliczenia miesięczne.
- Rejestr operacji i bezpieczeństwo – dziennik zmian danych, role użytkowników, kontrola dostępu.

2.1. Opis działania i schemat logiczny systemu

System wspiera pracę apteki lokalnej, w której występują dwie główne role użytkowników:

- **Kierownik apteki** – zarządza danymi o lekach gotowych i recepturach, nadzoruje dostawy, zatwierdza zamówienia do hurtowni, kontroluje raporty oraz uprawnienia użytkowników.
- **Farmaceuta** – realizuje recepty (na leki gotowe i recepturowe), przygotowuje leki robione, aktualizuje dane magazynowe oraz wystawia dokumenty sprzedaży.

Logiczny model bazy danych obejmuje następujące obszary informacyjne:

- **Leki gotowe** – dane o produktach leczniczych, ich producentach, partiach, terminach ważności oraz ilościach magazynowych.
- **Surowce recepturowe** – informacje o substancjach czynnych i pomocniczych używanych do sporządzania leków robionych wraz z ich zapasami.
- **Receptury leków robionych** – definicje składu, przeliczniki gramatury oraz koszty przygotowania poszczególnych leków recepturowych.
- **Rejestr realizowanych recept** – zapisy wszystkich zrealizowanych recept, zarówno na leki gotowe, jak i recepturowe (powiązanie z klientem, farmaceutą i wydаныmi produktami).
- **Rejestr klientów apteki** – dane klientów przechowywane zgodnie z wymogami RODO.
- **Magazyn** – obsługa dostaw, korekt stanów, rezerwacji, stanów minimalnych oraz zużycia surowców w procesie recepturowym.
- **Zamówienia do hurtowni** – zamówienia automatyczne i ręczne dla leków gotowych oraz surowców recepturowych.
- **Raporty i rozliczenia** – dane potrzebne do raportów sprzedaży, stanów magazynowych, zużycia składników oraz rozliczeń okresowych.
- **Rejestr operacji i bezpieczeństwo** – dziennik operacji użytkowników, role, uprawnienia i mechanizmy kontroli dostępu.

System działa lokalnie na komputerze apteki. Aplikacja komunikuje się z bazą PostgreSQL.

2.2. Wymagania funkcjonalne

(Kierownik apteki posiada wszystkie funkcje dostępne dla farmaceuty.)

F01) Zarządzanie danymi klientów (Create / Read / Update / Delete)

- Farmaceuta może tworzyć nowe wpisy klientów oraz przeglądać dane potrzebne do realizacji recept.
- Kierownik może dodatkowo aktualizować i usuwać dane klientów w ramach nadzoru administracyjnego.

F02) Realizacja recept na leki gotowe (Create / Read / Update)

- Farmaceuta tworzy wpis realizacji recepty, odczytuje dane leków i partii oraz aktualizuje stan magazynowy poprzez rejestrację wydania.
- Kierownik ma dostęp do przeglądania oraz korekty zrealizowanych recept.

F03) Realizacja recept recepturowych (Create / Read / Update)

- Farmaceuta tworzy zapis wykonania leku recepturowego, odczytuje skład receptury oraz aktualizuje ilości surowców zużytych w procesie.
- Kierownik może przeglądać oraz zatwierdzać receptury i ich realizacje.

F04) Rejestracja dostaw (Create / Read / Update / Delete)

- Farmaceuta tworzy rekordy dostaw, odczytuje dane o partiach oraz aktualizuje ilości magazynowe.
- Kierownik ma możliwość edycji i usuwania błędnych wpisów dotyczących dostaw.

F05) Aktualizacja stanów magazynowych (Read / Update)

- Farmaceuta odczytuje aktualne stany magazynowe i wykonuje ich aktualizację po dostawach i wydaniach.
- Kierownik posiada możliwość wprowadzania korekt stanów magazynowych.

F06) Korekty i usuwanie błędnych wpisów magazynowych (Update / Delete)

- Kierownik aktualizuje i usuwa wpisy magazynowe związane z przeterminowaniem, uszkodzeniem lub błędnym wprowadzeniem danych.
- System rejestruje wszystkie operacje korekt w dzienniku audytu.

F07) Obsługa zamówień do hurtowni (Create / Read / Update / Delete)

- Farmaceuta może tworzyć propozycje zamówień i odczytywać dane o zapasach.
- Kierownik przegląda, edytuje, zatwierdza i archiwizuje zamówienia, a także usuwa błędne pozycje.

F08) Generowanie raportów (Read)

- Kierownik odczytuje dane sprzedażowe, magazynowe i recepturowe oraz generuje raporty okresowe.
- Raporty mogą być eksportowane do zewnętrznych formatów (np. plików zewnętrznych).

2.3. Wymagania niefunkcjonalne

2.3.1. Wykorzystywane technologie i narzędzia

- System bazodanowy: **PostgreSQL**.
- Aplikacja kliencka: **C#** (aplikacja desktopowa) z interfejsem graficznym opartym o **Avalonia UI**.
- Środowisko pracy: komputery z systemem **Windows 10/11** lub **Linux (kernel ≥ 6.4.8)**.
- Minimalne wymagania sprzętowe:
 - procesor: co najmniej Intel Core i3 3300 (zalecany i5 5300 lub nowszy),
 - pamięć RAM: co najmniej 4 GB (zalecane 8 GB),
 - dysk: minimum 5 GB wolnego miejsca (zalecane ≥ 10 GB na kopie zapasowe).

2.3.2. Wymagania dotyczące rozmiaru bazy danych

System jest przeznaczony dla apteki obsługującej średnio **40–60 klientów dziennie**. W ciągu dnia realizowanych jest ok. **20–30 recept**, z czego część dotyczy leków recepturowych. Baza danych obejmuje:

- kilka tysięcy rekordów leków gotowych i surowców recepturowych,
- powiązane z nimi partie (dostawy, daty ważności),
- rejestr realizowanych recept, dokumentów sprzedaży i operacji magazynowych,
- rejestr klientów i użytkowników oraz dziennik operacji (log audytowy).

System musi zapewniać płynną pracę przy takim rozmiarze danych i jednoczesnej obsłudze przez 1–2 użytkowników.

2.3.3. Wymagania dotyczące bezpieczeństwa systemu

Wymagania niefunkcjonalne w zakresie bezpieczeństwa i jakości danych obejmują:

- **N01 – Bezpieczeństwo dostępu:**
System musi zapewniać mechanizmy logowania, role i uprawnienia użytkowników oraz blokować nieautoryzowany dostęp.
- **N02 – Szyfrowanie i ochrona danych:**
Dane muszą być zabezpieczone zgodnie z wymogami RODO; wymagane jest szyfrowanie i kontrola dostępu do danych wrażliwych (dane klientów/pacjentów).
- **N04 – Niezawodność:**
System powinien działać bez przerw i być odporny na typowe błędy użytkownika oraz awarie oprogramowania.
- **N05 – Kopie zapasowe:**
System wykonuje regularne kopie zapasowe oraz umożliwia szybkie odtworzenie danych po awarii.
- **N06 – Wydajność:**
System musi szybko wykonywać operacje związane ze sprzedażą, wyszukiwaniem danych (np. leków, recept, klientów) oraz obsługą dostaw.
- **N08 – Jakość danych:**
System waliduje dane wejściowe i zapewnia spójność informacji (np. poprawność dat, ilości, powiązań między encjami).
- **N10 – Alerty i monitoring:**
System wysyła powiadomienia o błędach, niskich stanach magazynowych oraz zbliżających się terminach ważności leków.

Dodatkowo system działa zgodnie z prawem farmaceutycznym, zasadami realizacji recept oraz przepisami RODO. Dane pacjentów przetwarzane są lokalnie przez Kierownika Apteki jako administratora danych.

2.4. Przyjęte założenia projektowe

Przyjęto następujące założenia:

1. Charakter systemu

- a. System jest lokalny, działa na komputerze apteki i nie wymaga stałego połączenia z Internetem.

- b. Baza danych znajduje się na lokalnym serwerze (lub tym samym komputerze), do którego łączy się aplikacja desktopowa.

2. Organizacja pracy apteki

- a. Apteka jest pojedynczą placówką o niewielkiej skali działania (40–60 klientów dziennie).
- b. W danym momencie z systemu korzysta 1–2 użytkowników (kierownik, farmaceuta).

3. Zgodność z przepisami

- a. Działanie systemu jest zgodne z prawem farmaceutycznym oraz zasadami realizacji recept.
- b. Przetwarzanie danych klientów odbywa się zgodnie z RODO; apteka pełni rolę administratora danych pacjentów.
- c. System zastępuje dokumentację papierową, z zachowaniem wymaganych okresów archiwizacji danych.

4. Bezpieczeństwo i audyt

- a. Wszystkie operacje użytkowników (kto, co, kiedy) są rejestrowane w dzienniku audytu.
- b. Wydanie leku wymaga poprawnej weryfikacji recepty, klienta oraz ważności partii leku.
- c. Leki recepturowe są wykonywane wyłącznie na podstawie zatwierdzonych receptur zapisanych w bazie.

5. Zakres funkcjonalny i techniczny

- a. System obejmuje pełny cykl życia leku w aptece: od dostawy i magazynowania, przez realizację recept, po raporty i rozliczenia.
- b. Aplikacja kliencka jest tworzona w C# z wykorzystaniem Avalonia UI, a dane przechowywane są w relacyjnej bazie PostgreSQL.
- c. Logiczny i fizyczny model bazy danych będzie rozwijany zgodnie z metodyką projektowania relacyjnych baz danych wymaganą w ramach przedmiotu.

3. Projekt systemu

W ramach projektu zaprojektowano i zaimplementowano jeden spójny model bazy danych w pgModeler, który jednocześnie pełni rolę **modelu konceptualnego, logicznego i fizycznego**. Ten sam schemat zawiera:

- identyfikację encji i związków (poziom konceptualny),
- relacyjny model tabel, kluczy i związków 1:N / N:M (poziom logiczny),
- konkretne typy danych, ograniczenia, sekwencje, widoki, funkcje, triggerzy i role (poziom fizyczny).

Poniżej opisano szczegółowo, w jaki sposób ten model spełnia wymagania każdego z poziomów.

3.1. Projekt bazy danych

3.1.1. Analiza rzeczywistości i uproszczony model konceptualny

Rzeczywistość, którą odwzorowuje system, to praca lokalnej apteki:

- obsługa klientów, lekarzy i użytkowników systemu (farmaceuci, kierownik),
- sprzedaż leków gotowych i recepturowych na podstawie recept,

- magazynowanie leków i surowców, kontrola partii i terminów ważności,
- dostawy i zamówienia do hurtowni,
- raportowanie i rozliczenia,
- bezpieczeństwo danych i rejestr operacji.

Na poziomie konceptualnym wyróżniono następujące główne encje i zależności:

Encje „osobowe” i adresowe

- **Osoba** – podstawowa encja reprezentująca człowieka (id_osoby, imię, nazwisko, numer_telefonu).
- **Klient** – specjalizacja osoby (id_klienta, PESEL, powiązanie z osobą i adresem).
- **Lekarz** – specjalizacja osoby (id_lekarza, numer_PWZ, powiązanie z osobą).
- **Użytkownik systemu** – specjalizacja osoby (id_uzytkownika, login, hasło, rola).
- **Adres** – encja wspólna dla klientów i dostawców (ulica, nr domu, nr lokalu, kod pocztowy, miasto, kraj).
- **Rola użytkownika** – encja definiująca typ dostępu (kierownik, farmaceuta).

Relacje (uproszczone):

- Osoba 1:1 Klient
- Osoba 1:1 Lekarz
- Osoba 1:1 Użytkownik
- Klient N:1 Adres
- Dostawca N:1 Adres

Encje związane z asortymentem

- **Lek** – produkt leczniczy dostępny w aptece (nazwa, kod EAN, producent, informacje merytoryczne).
- **Wariant leku** – konkretna postać i dawkowanie leku (tabletki, kapsułki, syrop...), powiązany z lekiem.
- **Producent** – wytwórca leku.
- **Surowiec** – substancja używana w recepturze aptecznej (typ, jednostka domyślna).
- **Partia leku** – konkretna partia magazynowa (numer partii, data ważności, ilość dostępna, ilość zarezerwowana).
- **Partia surowca** – analogicznie dla surowców.

Relacje:

- Wariant leku N:1 Lek
- Lek N:1 Producent
- Partia leku N:1 Lek
- Partia surowca N:1 Surowiec

Encje związane z magazynem i zaopatrzeniem

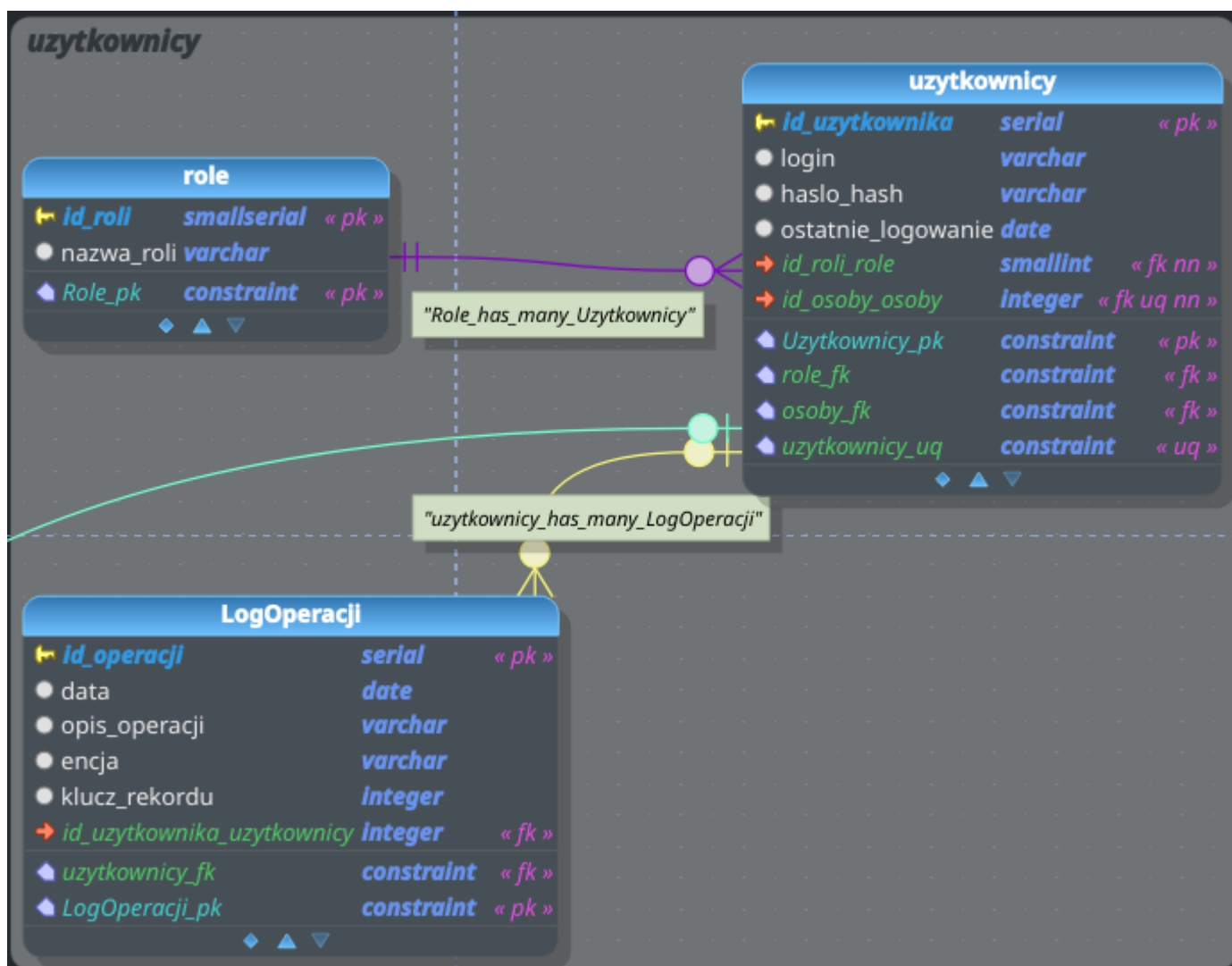
- **Dostawca** – kontrahent dostarczający leki i surowce.
- **Dostawa** – zdarzenie przyjęcia towaru (data, dostawca).
- **Zamówienie** – zamówienie do hurtowni (data złożenia, data realizacji, status, dostawca).
- **Relacje N:M surowców i dostawców** – umożliwiają wskazanie, który dostawca oferuje jakie surowce.

Encje związane z receptami i sprzedażą

- **Recepta** – dokument medyczny wystawiony dla klienta przez lekarza (data wystawienia, data realizacji).
- **Sprzedaż** – dokument sprzedaży w aptece (paragon/faktura).
- **Pozycja sprzedaży** – pojedyncza pozycja na dokumencie sprzedaży (ilość, cena jednostkowa, powiązanie z lekami lub recepturami).
- **Receptura** – definicja leku recepturowego (nazwa, opis).
- **Relacje N:M:**
 - Leki ↔ Recepty (jakie leki przepisano na receptach),
 - Pozycje sprzedaży ↔ Leki,
 - Pozycje sprzedaży ↔ Receptury (zeregrowane powiązania dla leków gotowych i recepturowych).

Encje związane z bezpieczeństwem

- **LogOperacji** – dziennik audytu zmian (data, typ operacji, encja, klucz rekordu, użytkownik).



Rys. 3.1. Fragment modelu: tabele ról i użytkowników oraz dziennik LogOperacji (normalizacja i audyt).

Taki konceptualny podział jest wprost odzwierciedlony w jednym projekcie pgModeler – każda encja ma odpowiadającą jej tabelę, a związki są zrealizowane jako klucze obce lub tabele N:M.

3.1.2. Model logiczny i normalizacja

Model logiczny to relacyjna postać powyższych encji – czyli konkretne tabele, kolumny i klucze. W projekcie:

- **każda encja konceptualna** została odwzorowana jako **tabela** (np. Osoba → apteka.osoby, Klient → apteka.klienci, Lek → apteka.leki),
- **każdy związek 1:N** został odwzorowany jako **klucz obcy** (np. Klient → Adres, Partia leku → Lek),
- **związki N:M** (np. Lek ↔ Recepty) jako **osobne tabele łączące** (many_leki_has_many_recepta, many_pozycja_sprzedarzy_has_many_leki, itd.).

Normalizacja – dlaczego model spełnia założenia 3NF

Najważniejsze decyzje normalizacyjne:

1. Osoby i role (klient, lekarz, użytkownik)

- a. Dane osobowe (imię, nazwisko, numer telefonu) trzymane są w jednej tabeli osoby.
- b. Klienci, lekarze i użytkownicy odwołują się do osoby po kluczu obcym.
- c. Eliminuje to powielanie imienia/nazwiska w wielu tabelach.
- d. Zależności funkcjonalne są jednoznaczne: atrybuty osoby zależą tylko od id_osoby, a atrybuty specyficzne dla klienta (PESEL), lekarza (PWZ) czy użytkownika (login, hasło, rola) zależą od ich własnych kluczy.

2. Adresy znormalizowane w osobnej tabeli

- a. Tabela adresy zawiera dane adresowe, a klienci i dostawcy przechowują tylko klucz obcy do adresu.
- b. Gdy ten sam klient ma wiele dokumentów, adres nie jest powielany.
- c. Zmiana adresu nie wymaga aktualizacji wielu wierszy – wystarczy jedna zmiana w tabeli adresy.

3. Leki i warianty leków

- a. Tabela leki przechowuje cechy wspólne produktu leczniczego (nazwa, kod EAN, producent, substancja czynna).
- b. Tabela warianty_lekow przechowuje cechy zależne od wariantu (postać, dawkowanie) i jest powiązana z leki.
- c. Dzięki temu unikasz powielania nazwy leku i producenta dla każdego wariantu.

4. Partie leków i surowców

- a. partie_lekow i partie_surowcow przechowują atrybuty typowe dla konkretnej partii (numer partii, data ważności, ilość dostępna, rezerwacje).
- b. Każda partia wskazuje na jeden lek / surowiec (1:N).
- c. Ilości i daty ważności nie są trzymane w leki / surowce – czyli nie łamią 3NF.

5. Recepty, sprzedaż i pozycje sprzedaży

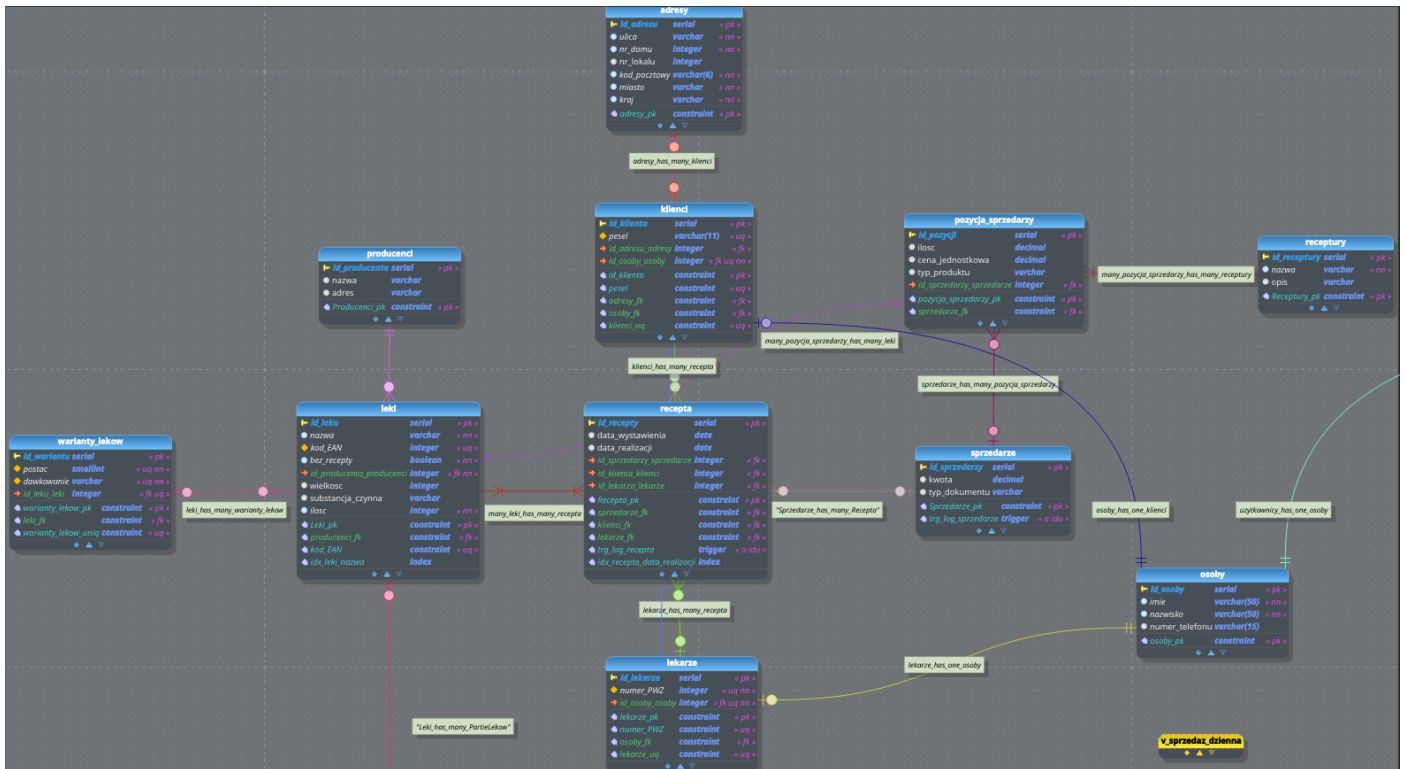
- a. recepta ma odwołania do klienta, lekarza i dokumentu sprzedaży – brak powtarzania danych osobowych.
- b. sprzedarzy opisują nagłówek dokumentu (kwota, typ, data),
- c. pozycja_sprzedarzy zawiera ilość, cenę jednostkową i powiązanie z dokumentem.
- d. Powiązanie pozycji z konkretnymi lekami/recepturami realizowane jest przez tabele N:M, co eliminuje zależności częściowe i nadmiar.

6. Unikalność i eliminacja duplikatów

- a. W kluczowych miejscach wprowadzono **ograniczenia UNIQUE** (PESEL, numer PWZ, kod EAN itp.), dzięki czemu:

- i. nie ma dwóch pacjentów z tym samym PESEL,
- ii. nie ma dwóch lekarzy z tym samym PWZ,
- iii. nie ma zduplikowanych produktów o tym samym kodzie EAN.

W rezultacie **model logiczny jest znormalizowany** – nie ma powtarzających się grup, atrybuty zależą od pełnego klucza i tylko od niego, a dane są podzielone na logiczne encje, bez niepotrzebnych powtórzeń.



Rys. 3.2. Częściowy diagram modelu bazy w pgModeler (schematy: apteka / magazyn / użytkownicy).

3.1.3. Model fizyczny i ograniczenia integralności danych

Model fizyczny to dokładne odwzorowanie modelu logicznego w konkretnej implementacji PostgreSQL. W projekcie:

Struktura schematów

- Schemat **apтека** – dane biznesowe związane z klientami, lekarzami, receptami, lekami, sprzedażą.
- Schemat **magazyn** – dane magazynowe (dostawy, zamówienia, partie leków i surowców, dostawcy).
- Schemat **uzytkownicy** – dane o użytkownikach systemu, rolach oraz dzienniku operacji.

Taki podział pozwala porządkować dane według obszarów funkcjonalnych i ułatwia nadawanie uprawnień.

Klucze główne i sekwencje

- Każda tabela posiada **klucz główny (PRIMARY KEY)** – zwykle typu serial (autoinkrementacja).
- Dla kluczy głównych tworzone są automatycznie **sekwencje**, zapewniające unikalność identyfikatorów.

Przykłady:

- apteka.klienci(id_klienta serial PRIMARY KEY, ...)
- magazyn.partie_lekow(id_partii serial PRIMARY KEY, ...)
- uzytkownicy.uzytkownicy(id_uzytkownika serial PRIMARY KEY, ...)

Klucze obce i spójność referencyjna

Związki między encjami odwzorowano jako **klucze obce (FOREIGN KEY)**:

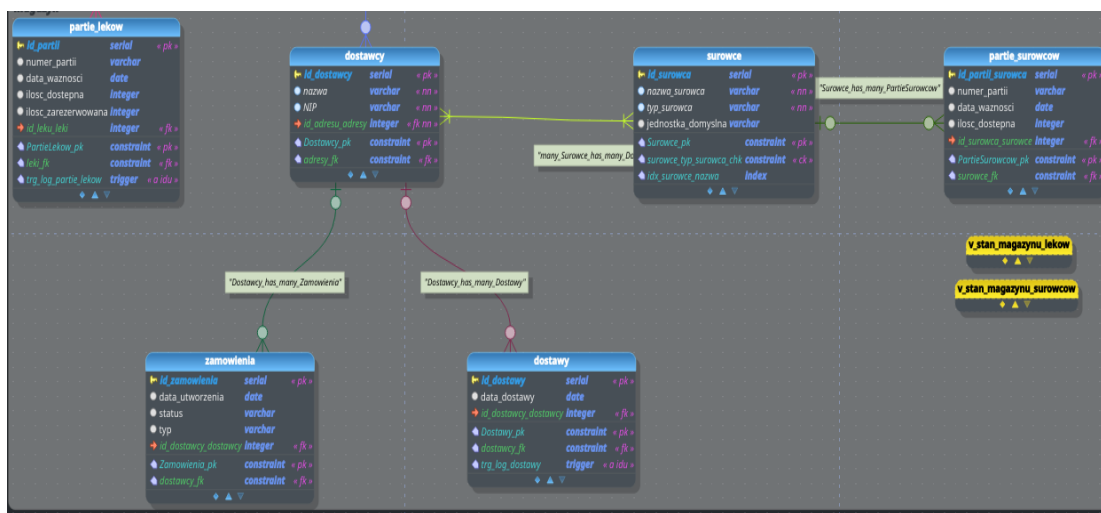
- klienci.id_osoby_osoby → osoby.id_osoby
- klienci.id_adresu_adresy → adresy.id_adresu
- lekarze.id_osoby_osoby → osoby.id_osoby
- leki.id_producenta_producenci → producenci.id_producenta
- partie_lekow.id_leku_leki → leki.id_leku
- partie_surowcow.id_surowca_surowce → surowce.id_surowca
- dostawy.id_dostawcy_dostawcy → dostawcy.id_dostawcy
- zamowienia.id_dostawcy_dostawcy → dostawcy.id_dostawcy
- recepta.id_klienta_klienci → klienci.id_klienta
- recepta.id_lekarza_lekarze → lekarze.id_lekarza
- recepta.id_sprzedarzy_sprzedarze → sprzedarze.id_sprzedarzy
- pozycja_sprzedarzy.id_sprzedarzy_sprzedarze → sprzedarze.id_sprzedarzy
- tabele N:M łączące leki/receptury z receptami i pozycjami sprzedaży mają FK i do obu stron relacji,
- uzytkownicy.uzytkownicy.id_rol_i_role → role.id_rol_i,
- LogOperacji.id_uzytkownika_uzytkownicy → uzytkownicy.id_uzytkownika.

Dzięki temu **nie można wstawić „osieroconych” rekordów**, np.:

- recepty bez istniejącego klienta,
- partii surowca bez powiązanego surowca,
- użytkownika bez roli,
- wpisu w dzienniku bez użytkownika.

Ograniczenia NOT NULL, UNIQUE, CHECK

- **NOT NULL** – w kluczowych kolumnach wymagających wartości (identyfikatory, nazwy, PESEL, numer PWZ, daty) zabezpiecza przed wprowadzeniem niekompletnych danych.
- **UNIQUE** – wymusza unikalność m.in. na:
 - PESEL w klienci,
 - numerze PWZ w lekarze,
 - kodzie EAN w leki,
 - loginie w uzytkownicy.
- **CHECK** – tam, gdzie to potrzebne (np. typ surowca, status zamówienia), pozwala ograniczyć wartości do dopuszczalnego zbioru.



Rys. 3.3. Fragment modelu: obiekty wykorzystywane przez widoki raportowe i ich powiązania.

Model fizyczny spełnia założenia:

- **logiczną poprawność** (zgodność z modelem logicznym),
- **spójność referencyjną** (FOREIGN KEY),
- **poprawność domenową** (NOT NULL, typy danych, ograniczenia UNIQUE i CHECK),
- **jednoznaczność identyfikacji wierszy** (PRIMARY KEY + sekwencje).

3.1.4. Inne elementy schematu – mechanizmy przetwarzania danych

Oprócz tabel projekt zawiera mechanizmy przetwarzania danych w samej bazie:

Widoki (VIEW)

Widoki służą do przygotowania danych raportowych i uproszczenia zapytań:

- **magazyn.v_stan_magazynu_lekow** – agreguje informacje o stanie magazynowym leków:
 - sumuje ilości dostępne i zarezerwowane w partiach,
 - pokazuje najbliższą datę ważności dla danego leku.
- **magazyn.v_stan_magazynu_surowcow** – analogicznie dla surowców recepturowych.
- **apteka.v_sprzedaz_dzienna** – wylicza dzienny obrót:
 - korzysta z daty sprzedaży i pozycji sprzedaży (ilość × cena jednostkowa),
 - grupuje dane po dacie.

Widoki „zamrażają” logikę raportów po stronie bazy, więc aplikacja kliencka może korzystać z prostych SELECT-ów typu `SELECT * FROM v_stan_magazynu_lekow`.

View properties



General



SQL definition



References



Columns



Triggers



Rules



Unnecessary semicolon(s) found at the end of the definition command! They will be ignored when generating the view's SQL code.



Load



Save



Copy



Edit



Search



Wrap



Clear

SELECT

```
s.data_sprzedazy AS data,  
SUM(ps.ilosc * ps.cena_jednostkowa) AS obrot  
FROM apteka.sprzedarze AS s  
JOIN apteka.pozycja_sprzedarzy AS ps  
ON ps.id_sprzedarzy_sprzedarze = s.id_sprzedarzy  
GROUP BY  
s.data_sprzedazy  
ORDER BY  
s.data_sprzedazy;
```

pgModeler



Apply



Cancel

View properties



General



SQL definition



References



Columns



Triggers



Rules



Unnecessary semicolon(s) found at the end of the definition command! They will be ignored when generating the view's SQL code.



Load



Save



Copy



Edit



Search



Wrap



Clear

SELECT

```
    l.id_leku,  
    l.nazwa,  
    COALESCE(SUM(p.ilosc_dostepna - COALESCE(p.ilosc_zarezerwowana, 0)), 0)  
    MIN(p.data_waznosci) AS najblizsza_data_waznosci  
FROM apteka.leki AS l  
LEFT JOIN magazyn.partie_lekow AS p  
    ON p.id_leku_leki = l.id_leku  
GROUP BY  
    l.id_leku,  
    l.nazwa  
ORDER BY  
    l.nazwa;
```

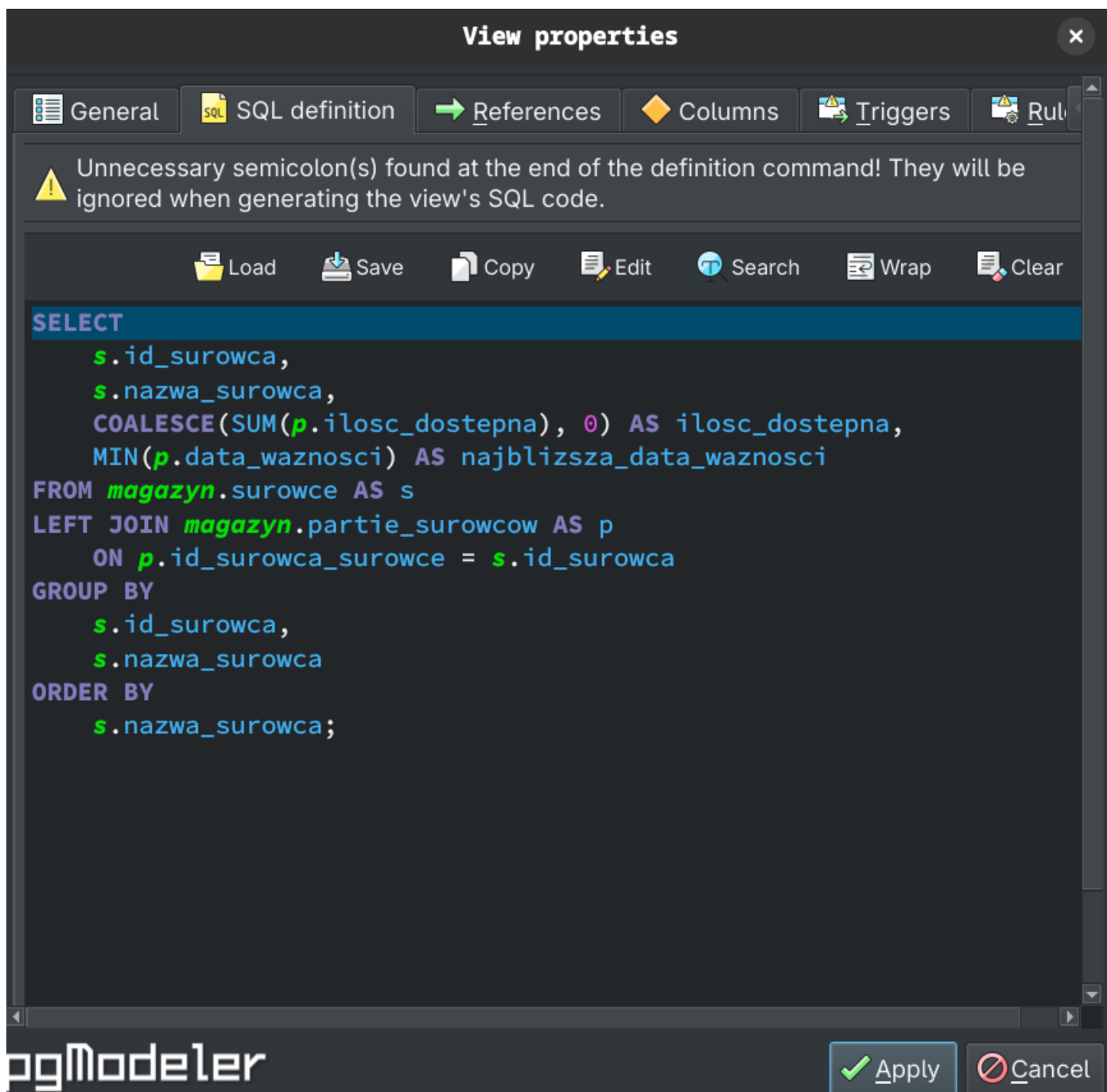
pgModeler



Apply



Cancel



Funkcje i triggerzy (logowanie operacji)

W schemacie **uzytkownicy** zdefiniowano mechanizmy audytu:

- **Funkcja fn_zapisz_log_operacji** – przyjmuje:
 - identyfikator użytkownika,
 - opis operacji (np. „INSERT on recepta”),
 - nazwę encji (tabeli),
 - klucz rekordu.Wstawia wpis do tabeli LogOperacji.
- **Funkcje triggerów** dla wybranych tabel (np. recepta, sprzedarze, dostawy, partie_lekow):
 - pobierają identyfikator użytkownika z ustawionej zmiennej sesyjnej (app.user_id),
 - budują opis operacji na podstawie TG_OP (INSERT/UPDATE/DELETE) i TG_TABLE_NAME,
 - wywołują fn_zapisz_log_operacji,
 - zwracają NEW lub OLD (zgodnie z wymaganiami triggerów).
- **Triggerzy** (obiekty TRIGGER) są powiązane z tabelami i uruchamiają funkcje:
 - BEFORE/AFTER INSERT, UPDATE, DELETE (w zależności od konfiguracji).

Dzięki temu każda istotna operacja na danych jest **automatycznie rejestrowana w dzienniku audytu**, bez konieczności dopisywania tej logiki w aplikacji C#.

Indexy

W naszej bazie danych apteki podstawowy zestaw indeksów zapewniają klucze główne (PRIMARY KEY). Dla każdej kolumny pełniącej rolę klucza głównego PostgreSQL automatycznie tworzy indeks, m.in. dla:

- leki.id_leku
- recepta.id_recepty
- dostawcy.id_dostawcy
- dostawy.id_dostawy
- partie_lekow.id_partii
- surowce.id_surowca
- zamowienia.id_zamowienia
- producenci.id_producenta
- sprzedarzy.id_sprzedarzy
- receptury.id_receptury
- pozycja_sprzedarzy.id_pozycji
- klienci.id_klienta
- lekarze.id_lekarza
- warianty_lekow.id_wariantu
- adresy.id_adresu
- numery_telefonu.id_telefonu
- uzytkownicy.id_uzytkownika
- LogOperacji.id_operacji

Dzięki tym indeksom system może bardzo szybko odnajdywać konkretne rekordy na podstawie ich identyfikatorów technicznych (np. pobranie leku po id_leku, sprzedaży po id_sprzedarzy czy pacjenta po id_klienta).

Dodatkowo w projekcie wykorzystywane są indeksy wynikające z ograniczeń unikalności (UNIQUE). PostgreSQL tworzy dla nich indeksy unikalne, na przykład:

- leki.kod_EAN – unikalny kod kreskowy leku (EAN); indeks pozwala szybko odnaleźć lek po zeskanowanym kodzie oraz uniemożliwia wprowadzenie duplikatu,
- klienci.pesel – unikalny numer PESEL pacjenta; indeks przyspiesza wyszukiwanie pacjenta po PESEL-u i wymusza brak powtórzeń,
- lekarze.numer_PWZ – unikalny numer prawa wykonywania zawodu lekarza; indeks umożliwia szybkie sprawdzenie, czy lekarz o danym numerze PWZ znajduje się w systemie.

Oprócz indeksów wynikających z kluczy głównych i ograniczeń unikalności, w projekcie zdefiniowano także indeksy nieunikalne na kolumnach często używanych w typowych operacjach biznesowych:

- indeks „idx_leki_nazwa” na kolumnie leki.nazwa – przyspiesza wyszukiwanie leków po nazwie, co jest standardową operacją podczas obsługi pacjenta,
- indeks „idx_surowce_nazwa” na kolumnie surowce.nazwa_surowca – umożliwia szybkie wyszukiwanie surowców według nazwy, co jest istotne przy pracy z magazynem surowców do produkcji leków recepturowych,

- indeks „idx_recepta_data_realizacji” na kolumnie recepta.data_realizacji – przyspiesza raporty oraz filtrowanie recept w określonych przedziałach czasowych (np. zestawienia sprzedaży, kontrola realizacji recept w danym miesiącu).

3.1.5. Projekt mechanizmów bezpieczeństwa na poziomie bazy danych

Bezpieczeństwo jest realizowane zarówno na poziomie logiki aplikacji, jak i **poziomu bazy danych**.

Role i uprawnienia (kontrola dostępu)

W bazie zdefiniowano role odzwierciedlające role biznesowe:

- **rola_kierownik** – kierownik apteki,
- **rola_farmaceuta** – farmaceuta.

Uprawnienia zostały nadane w sposób zgodny z wymaganiami systemu:

- Role mają **CONNECT** do bazy oraz **USAGE** na odpowiednich schematach.
- **Farmaceuta:**
 - może odczytywać i modyfikować dane niezbędne do bieżącej pracy (recepty, sprzedaż, magazyn, klienci),
 - ma dostęp SELECT do widoków raportowych,
 - ma ograniczony dostęp do wrażliwych tabel (np. użytkownicy, pełny log operacji tylko w razie potrzeby).
- **Kierownik:**
 - ma szerszy zakres uprawnień – pełny CRUD tam, gdzie wymagają tego funkcje administracyjne,
 - może przeglądać log operacji,
 - ma dostęp do mechanizmów raportowania i korekt danych (np. korekta stanów magazynowych, usuwanie błędnych dostaw).

Uprawnienia są nadawane na poziomie:

- **tabel** (SELECT, INSERT, UPDATE, DELETE),
- **widoków** (SELECT dla raportów),
- **funkcji** (EXECUTE – np. do logowania).

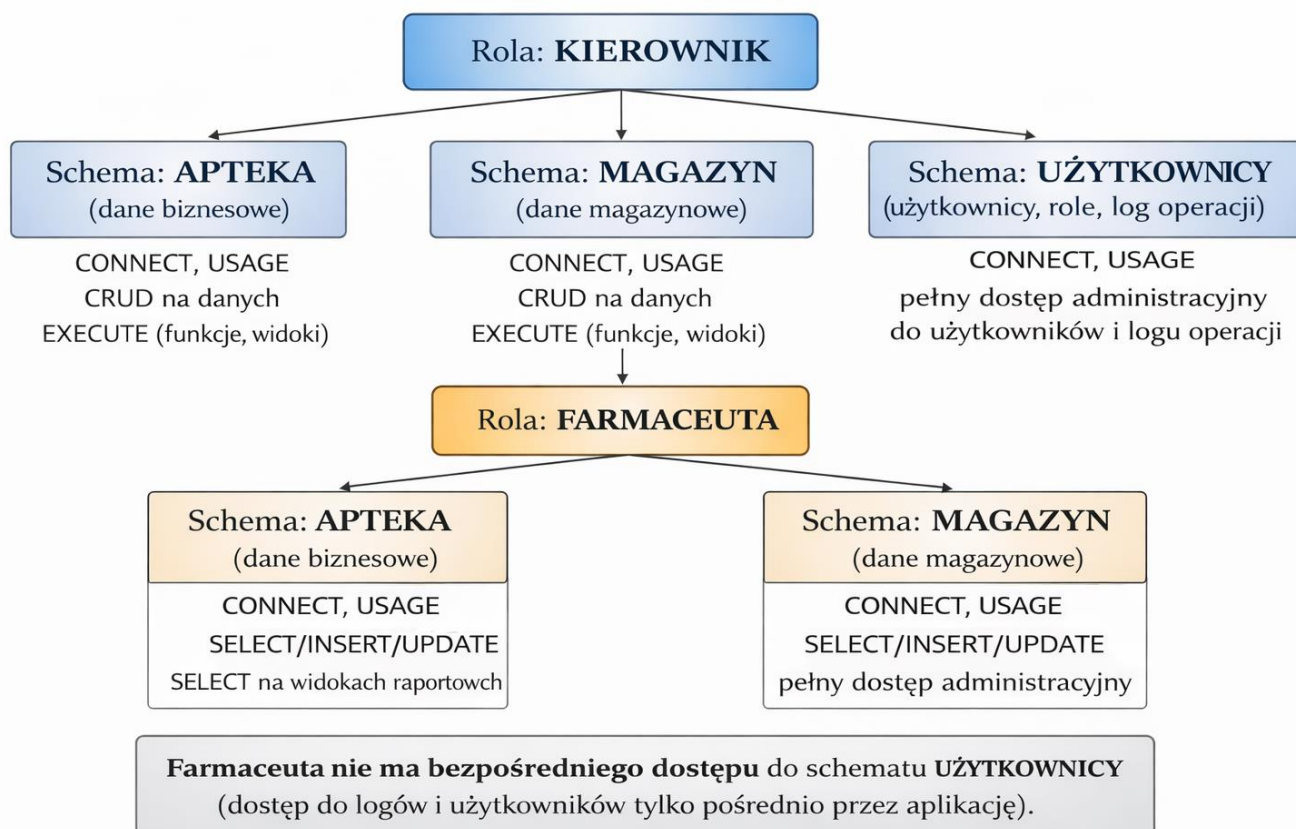
Dzięki temu **rola użytkownika w bazie** odpowiada dokładnie jego roli w systemie biznesowym.

Bezpieczeństwo danych osobowych i audyt

- Dane osobowe klientów/lekarzy są zorganizowane w osobnych tabelach (osoby, klienci, lekarze), co pozwala precyzyjniej kontrolować dostęp na poziomie tabel.
- Dziennik LogOperacji przechowuje:
 - kto (id_uzytkownika),
 - co (opis operacji, encja),
 - kiedy (data),

- na którym rekordzie (klucz rekordu).
- Mechanizmy triggerów **uniemożliwiają pominięcie logowania** – każda zmiana w tabelach objętych audytem będzie zapisana.

Diagram Bezpieczeństwa:



Rys. 3.7. Diagram bezpieczeństwa: role (kierownik/farmaceuta) i zakres uprawnień do schematów bazy.

Jednolity model a spełnienie wymagań logicznych i fizycznych

Choć projekt jest przygotowany w jednym modelu w pgModeler, spełnia on wymogi:

- **Modelu logicznego:**
 - poprawny podział na encje i relacje,
 - znormalizowane tabele bez nadmiarowych zależności,
 - relacje 1:N i N:M odwzorowane jako FK i tabele pośrednie,
 - jednoznaczne klucze główne i ograniczenia unikalności.
- **Modelu fizycznego:**
 - zdefiniowane schemy (apteka, magazyn, uzytkownicy),
 - konkretne typy danych (integer, varchar, date, decimal, boolean, itp.),
 - wszystkie klucze obce i ich zachowanie przy aktualizacji/usuwaniu,
 - sekwencje i indeksy dla PK i UNIQUE,
 - widoki, funkcje i triggery,
 - role i nadane im uprawnienia (GRANT/REVOKE).

W efekcie:

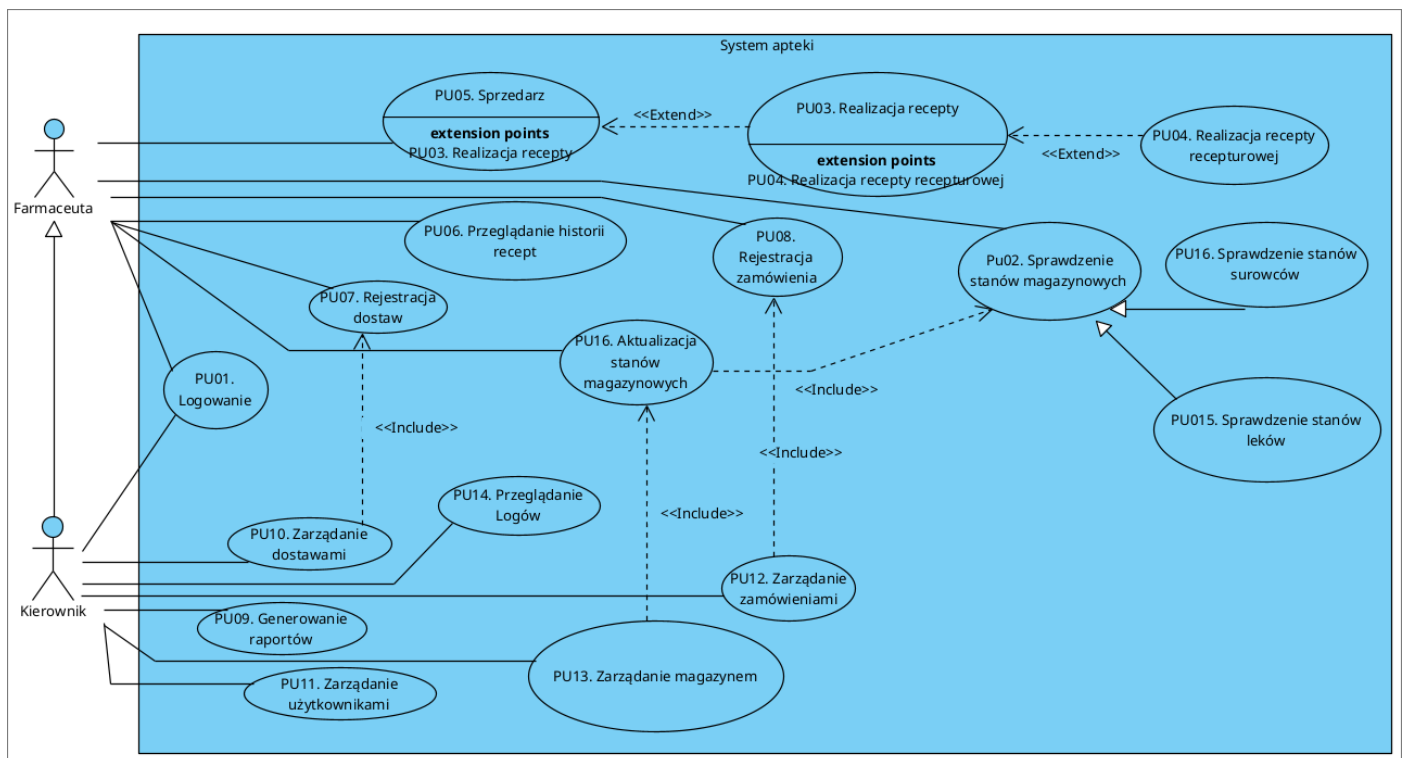
- **Na poziomie logicznym** – model poprawnie opisuje rzeczywistość apteki i eliminuje redundancję danych.

- **Na poziomie fizycznym** – jest kompletnym schematem PostgreSQL, gotowym do wdrożenia: zapewnia spójność, wydajność (indeksy na PK/UNIQUE), bezpieczeństwo i mechanizmy operacyjne (widoki, triggerzy, audyt).

3.2. Projekt aplikacji użytkownika

Aplikacja użytkownika została zaprojektowana jako **desktopowa aplikacja w języku C#** z wykorzystaniem biblioteki **Avalonia UI**. Program pracuje lokalnie na komputerze apteki, łączy się bezpośrednio z bazą danych PostgreSQL i obsługuje dwóch typowych użytkowników: **kierownika apteki** oraz **farmaceutę**.

1. **Diagram przypadków użycia systemu**
2. Na rysunku przedstawiono **diagram przypadków użycia systemu zarządzania apteką**.
3. Diagram pokazuje zakres odpowiedzialności systemu oraz głównych aktorów, którzy z niego korzystają



Rys. 3.8. Diagram przypadków użycia systemu zarządzania apteką

3.2.1. Architektura aplikacji i diagramy projektowe

Aplikacja została zaprojektowana w oparciu o **trójwarstwową architekturę logiczną**:

1. Warstwa prezentacji (UI)

- a. Widoki i kontrolki Avalonia UI (okna: logowanie, okno główne, formularze szczegółowe).
- b. Prosta obsługa zdarzeń (kliknięcia przycisków, wybór z listy, walidacja pól).
- c. Brak bezpośredniego dostępu do bazy danych – UI komunikuje się wyłącznie z warstwą logiki.

2. Warstwa logiki biznesowej (services)

- a. Klasy realizujące scenariusze biznesowe: logowanie, realizacja recepty, rejestracja dostawy, aktualizacja stanów magazynowych, generowanie raportów.
- b. Odpowiada za:
 - i. sprawdzanie uprawnień użytkownika (rola kierownik/farmaceuta),
 - ii. walidację danych (np. poprawność PESEL, dat, ilości),
 - iii. wywołania operacji na warstwie dostępu do danych.

3. Warstwa dostępu do danych (DAL)

Open Database Connectivity (ODBC) to standardowy interfejs programistyczny (API) pozwalający aplikacjom łączyć się z różnymi systemami baz danych w jednolity sposób, niezależnie od konkretnego silnika (np. PostgreSQL, MySQL, SQL Server) i systemu operacyjnego. ODBC wprowadza warstwę pośrednią – aplikacja wywołuje funkcje ODBC, a specjalny sterownik ODBC tłumaczy te wywołania na odpowiednie zapytania SQL i protokół komunikacji obsługiwany przez dany system zarządzania bazą danych.

W naszym projekcie systemu zarządzania apteką ODBC może służyć jako warstwa dostępu do danych dla aplikacji desktopowej, która łączy się z relacyjną bazą PostgreSQL. Wystarczy zainstalować sterownik PostgreSQL ODBC (psqlODBC) i skonfigurować źródło danych (DSN), które wskazuje na naszą bazę. Dzięki temu kod aplikacji korzysta z jednego, standardowego interfejsu, a ewentualna zmiana środowiska (np. przeniesienie aplikacji na inny komputer) wymaga jedynie poprawnej konfiguracji ODBC, a nie modyfikacji logiki dostępu do danych. Rozwiązanie to dobrze pasuje do naszej architektury: aplikacja działa lokalnie (offline), komunikuje się z lokalną instancją PostgreSQL, a ODBC zapewnia prostą, przenośną i dobrze udokumentowaną metodę obsługi połączeń z bazą danych.

3.2.2. Interfejs graficzny i struktura menu

Interfejs graficzny został zaprojektowany jako **prost y i kafelkowy**, tak aby był czytelny dla pracowników apteki.

Ekran logowania

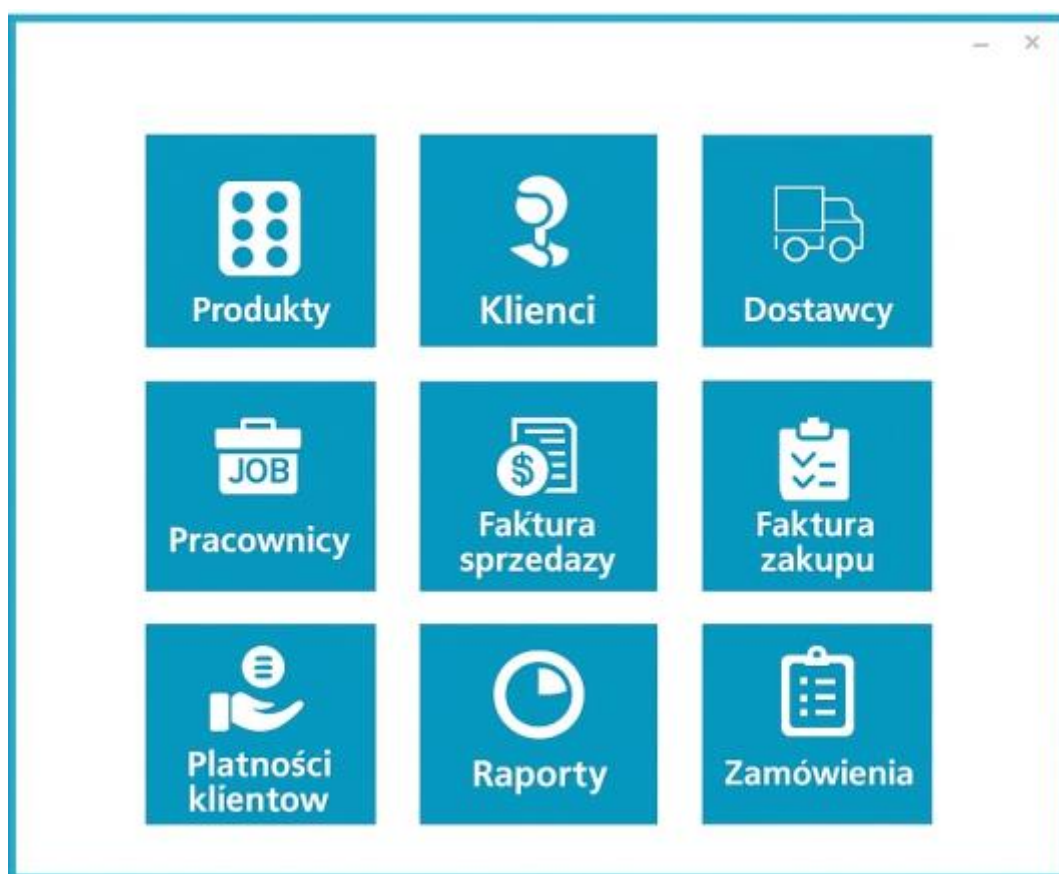
- Okno zawiera:
 - pole „**Nazwa użytkownika**”,
 - pole „**Hasło**” (maskowane),
 - przycisk „**Zaloguj**”.
- Po poprawnym zalogowaniu:
 - identyfikator użytkownika i jego rola pobierane są z tabeli `uzytkownicy.uzytkownicy`,
 - w sesji połączenia z bazą ustawiany jest parametr `app.user_id`,
 - wyświetlane jest okno główne.



Rys. 3.9. Projekt okna logowania aplikacji (login/hasło).

Formularz główny (menu kafelkowe)

Formularz główny składa się z siatki dużych kafelków reprezentujących moduły systemu. Przykładowe kafelki (zgodnie z załączonym zrzutem ekranu):



Rys. 3.10. Projekt formularza głównego – menu kafelkowe modułów aplikacji.

- **Produkty** – przejście do listy leków i wariantów leków.
- **Klienci** – zarządzanie danymi klientów apteki.
- **Dostawcy** – obsługa kontrahentów i powiązanych adresów.

- **Pracownicy** – podgląd użytkowników systemu (tylko kierownik).
- **Faktura sprzedaży** – wystawianie dokumentów sprzedaży na podstawie recept i wyboru produktów.
- **Faktura zakupu** – rejestracja dostaw i faktur od hurtowni.
- **Płatności klientów** – rozliczenia płatności, historia operacji finansowych.
- **Raporty** – dostęp do raportów sprzedaży, stanów magazynowych, zużycia surowców.
- **Zamówienia** – tworzenie, edycja i zatwierdzanie zamówień do hurtowni.

Zachowanie kafelków zależy od roli użytkownika:

- **Kierownik** widzi wszystkie kafelki i ma dostęp do wszystkich funkcji.
- **Farmaceuta** ma ograniczony dostęp – wybrane kafelki mogą być:
 - ukryte (np. „Pracownicy”),
 - dostępne tylko w trybie odczytu lub z ograniczoną funkcjonalnością (np. zamówienia, wybrane raporty).

W każdym module stosowany jest podobny układ:

- lista (tabelka) z możliwością filtrowania i wyszukiwania,
- przyciski operacji: „Dodaj”, „Edytuj”, „Usuń”, „Szczegóły”,
 - formularz szczegółowy z polami odpowiadającymi kolumnom w tabelach bazy danych.

Widok modułu (np. Klienci)

Szukaj:

ID	Imię i nazwisko	PESEL	Telefon
1	Jan Kowalski	90010112345	123 456 789
2	Anna Nowak	8803039876	987 654 321
...	1		

Formularz szczegółowy rekordu

ID:

Imię:

Nazwisko:

PESEL:

Telefon:

Adres:

- Rys. 3.11. Projekt widoku modułu (np. „Klienci”): lista rekordów + panel szczegółów + operacje CRUD.

3.2.3. Projekt wybranych funkcji systemu

Na rysunku X przedstawiono diagram przypadków użycia systemu apteki (PU01–PU16).

Poniżej opisano wybrane, kluczowe przypadki użycia z tego diagramu – wraz z przepływem danych pomiędzy aplikacją a bazą danych PostgreSQL.

3.2.3.1. PU01 – Logowanie użytkownika

Aktorzy:

- Farmaceuta
- Kierownik

Cel:

Uwierzytelnienie użytkownika, ustalenie jego roli (farmaceuta / kierownik) oraz przygotowanie sesji połączenia z bazą danych.

Przebieg podstawowy:

1. Użytkownik uruchamia aplikację i wybiera opcję „**Logowanie**” (przypadek użycia PU01).
2. W formularzu logowania wprowadza:
 - a. nazwę użytkownika,
 - b. hasło (pole maskowane).
3. Aplikacja wywołuje metodę logiki biznesowej, np.:
AuthenticationService.Login(login, haslo).
4. Warstwa logiki:
 - a. pobiera z bazy rekord użytkownika z tabeli uzytkownicy.uzytkownicy,
 - b. porównuje podane hasło z haszem zapisanym w bazie,
 - c. odczytuje przypisaną rolę użytkownika (kierownik / farmaceuta).
5. Po poprawnej autoryzacji:
 - a. zestawiane jest połączenie z bazą danych PostgreSQL,
 - b. wykonywane jest polecenie SQL:

SET app.user_id = <id_uzytkownika>;

co pozwala triggerom w bazie zapisywać operacje w tabeli LogOperacji,

- c. wyświetlane jest formularz główny z kafelkowym menu, w którym dostępne są przypadki użycia zgodne z rolą użytkownika (PU05 Sprzedaż, PU07 Rejestracja dostaw, PU08 Rejestracja zamówienia, PU09 Generowanie raportów, PU10–PU16 itd.).

Warunki końcowe:

- Użytkownik jest zalogowany, a system zna jego rolę i identyfikator.
- Możliwe jest wykonanie kolejnych przypadków użycia opisanych na diagramie.

3.2.3.2. PU05 / PU03 / PU04 – Sprzedaż i realizacja recepty

Na diagramie sprzedaż została podzielona na kilka powiązanych przypadków użycia:

- **PU05. Sprzedaż** – ogólny przypadek użycia obsługujący sprzedaż,
- **PU03. Realizacja recepty** – rozszerza PU05 (<<extend>>) w sytuacji sprzedaży na receptę,
- **PU04. Realizacja recepty recepturowej** – rozszerza PU03 (<<extend>>) w przypadku recepty na lek robiony.

3.2.3.2.1. PU05 – Sprzedaż

Aktor:

- Farmaceuta (Kierownik również, z szerszymi uprawnieniami)

Przebieg podstawowy:

1. Z formularza głównego użytkownik wybiera kafelek odpowiadający sprzedaży (np. „Faktura sprzedaży” / „Sprzedaż”) – uruchamiany jest przypadek użycia **PU05. Sprzedaż**.
2. System tworzy nowy dokument sprzedaży w pamięci i otwiera formularz:
 - a. umożliwia wyszukiwanie leków,
 - b. dodawanie pozycji sprzedaży,
 - c. powiązanie sprzedaży z klientem.
3. Jeżeli sprzedaż odbywa się na podstawie recepty, proces jest rozszerzany o przypadek **PU03. Realizacja recepty** (<<extend>>).
4. W sytuacji, gdy recepta dotyczy leku recepturowego, dodatkowo aktywowany jest przypadek **PU04. Realizacja recepty recepturowej** (<<extend>>).

3.2.3.2.2. PU03 – Realizacja recepty (leki gotowe)

Aktor:

- Farmaceuta

Przebieg podstawowy:

1. Farmaceuta wybiera w menu opcję „Recepty” lub odpowiednią funkcję w module sprzedaży, co uruchamia przypadek **PU03. Realizacja recepty**.
2. W formularzu wprowadza:
 - a. dane klienta (wybór z listy lub dodanie / edycja),
 - b. dane lekarza,
 - c. numer recepty oraz daty (wystawienia, realizacji),
 - d. listę przepisanych leków gotowych.
3. Aplikacja wykonuje operacje na bazie:
 - a. tworzy rekord w tabeli apteka.sprzedarze (nagłówek dokumentu sprzedaży),
 - b. dodaje rekord w tabeli apteka.recepta, powiązany z klientem i lekarzem,
 - c. dla każdej pozycji sprzedaży:
 - i. dodaje wiersz w tabeli apteka.pozycja_sprzedarzy,
 - ii. dopisuje powiązania w tabeli N:M many_pozycja_sprzedarzy_has_many_leki,
 - d. aktualizuje stany magazynowe poprzez zmniejszenie ilości dostępnej w magazyn.partie_lekow dla wykorzystanych partii.
4. Triggery w bazie wywołują funkcję logującą i dopisują wpis do tabeli LogOperacji (informacja kto, co i na jakim rekordzie zmienił).

3.2.3.2.3. PU04 – Realizacja recepty recepturowej

Aktor:

- Farmaceuta

Opis:

PU04 rozszerza przypadek PU03 w sytuacji, gdy recepta dotyczy leków recepturowych (robionych w aptece z surowców).

Przebieg podstawowy:

1. W trakcie realizacji recepty farmaceuta dodaje pozycję powiązaną z odpowiednią **recepturą** (tabela receptury).
2. Aplikacja, oprócz działań z PU03:
 - a. wiąże pozycję sprzedaży z recepturą w tabeli N:M
many_pozycja_sprzedarzy_has_many_receptury,
 - b. oblicza ilości surowców zużywanych do wykonania danej receptury,
 - c. aktualizuje ilości dostępne w magazynie partie_surowcow.
3. Sprzedaż recepturowa jest nadal częścią tego samego dokumentu sprzedaży (PU05), ale powoduje dodatkowe zmiany w magazynie surowców.

3.2.3.3. PU07 / PU10 / PU13 / PU16 – Rejestracja dostaw i aktualizacja stanów magazynowych

Proces przyjmowania dostaw oraz zarządzania magazynem został na diagramie odwzorowany poprzez kilka powiązanych przypadków użycia:

- **PU10. Zarządzanie dostawami** – nadrzędny przypadek obsługujący listę dostaw,
- **PU07. Rejestracja dostaw** – szczegółowy przypadek dodania nowej dostawy, <<include>> w PU10,
- **PU13. Zarządzanie magazynem** – szerszy przypadek zarządzania magazynem (partie, korekty),
- **PU16. Aktualizacja stanów magazynowych** – wspólny przypadek użycia do przeliczenia stanów, <<include>> w PU07, PU08, PU12 oraz PU13,
- **PU02 / PU015 / PU016 – Sprawdzenie stanów magazynowych, leków i surowców** – przypadki użycia służące odczytowi bieżących stanów.

3.2.3.3.1. PU07 – Rejestracja dostaw

Aktorzy:

- Kierownik
- Farmaceuta (w roli osoby obsługującej magazyn)

Przebieg podstawowy:

1. Użytkownik wybiera w formularzu głównym kafelek „**Dostawy / Faktura zakupu**”, co uruchamia przypadek **PU10. Zarządzanie dostawami**.
2. Z poziomu PU10 wybierana jest opcja „Nowa dostawa” – wykonywany jest przypadek <<include>> **PU07. Rejestracja dostaw**.
3. W formularzu dostawy użytkownik wprowadza:
 - a. dostawcę (powiązanie z tabelą magazyn.dostawcy),
 - b. datę dostawy i numer dokumentu,

- c. listę pozycji: lek/surowiec, numer partii, datę ważności, ilość, cenę zakupu.
4. Aplikacja zapisuje dane w bazie:
 - a. dodaje rekord w tabeli magazyn.dostawy,
 - b. dla każdej pozycji:
 - i. tworzy nową partię w tabeli magazyn.partie_lekow lub magazyn.partie_surowcow,
 - ii. lub aktualizuje istniejącą partię (zwiększenie ilości dostępnej).
5. Po zapisie dostawy wykonywany jest przypadek <<include>> **PU16. Aktualizacja stanów magazynowych**, który:
 - a. przelicza wartości wykorzystywane w raportach,
 - b. zapewnia spójność widoków v_stan_magazynu_lekow i v_stan_magazynu_surowcow.
6. Triggery w bazie rejestrują operację w LogOperacji.

3.2.3.4. PU09 – Generowanie raportów

Aktor:

- Kierownik

Cel:

Uzyskanie przekrojowych informacji o sprzedaży, stanach magazynowych i zużyciu surowców.

Przebieg podstawowy:

1. Kierownik z formularza głównego wybiera kafelek „**Raporty**”, co uruchamia przypadek **PU09. Generowanie raportów**.
2. W formularzu raportów wybiera:
 - a. typ raportu, np.:
 - i. sprzedaż dzienna,
 - ii. aktualny stan magazynu leków,
 - iii. aktualny stan magazynu surowców,
 - b. zakres dat,
 - c. ewentualne dodatkowe filtry (np. grupa leków, dostawca).
3. Aplikacja wykonuje odpowiednie zapytania do widoków bazy danych, m.in.:
 - a. apteka.v_sprzedaz_dzienna,
 - b. magazyn.v_stan_magazynu_lekow,
 - c. magazyn.v_stan_magazynu_surowcow.
4. Wyniki raportu prezentowane są w postaci tabelarycznej z możliwością:
 - a. sortowania i filtrowania,
 - b. eksportu do formatu zewnętrznego (np. CSV, PDF).
5. Na podstawie wygenerowanych raportów kierownik podejmuje decyzje dotyczące np. planowania zamówień (powiązanie z PU08 / PU12) oraz korekt magazynowych (PU13, PU16).

3.2.5. Projekt zabezpieczeń na poziomie aplikacji

Zabezpieczenia systemu są uzupełnieniem mechanizmów zaimplementowanych w bazie danych.

Autoryzacja i role

- Formularz logowania weryfikuje poprawność nazwy i hasła użytkownika na podstawie danych z tabeli użytkowników.
- Po zalogowaniu w aplikacji przechowywana jest informacja o **roli** (kierownik/farmaceuta).
- Rola użytkownika decyduje o:
 - dostępności poszczególnych kafelków w menu (np. moduł „Pracownicy” tylko dla kierownika),
 - dostępności operacji na formularzach (np. usuwanie dostawy tylko dla kierownika),
 - zakresie generowanych raportów.

Walidacja danych po stronie aplikacji

- Wszystkie formularze:
 - sprawdzają poprawność wymaganych pól (np. brak pustego PESEL, poprawny format daty),
 - ograniczają zakres możliwych wartości (np. ilości większe od zera),
 - prezentują użytkownikowi czytelne komunikaty o błędach.
- Dzięki temu do bazy danych trafiają już wstępnie zwalidowane dane, co zmniejsza ryzyko błędów logicznych.

Ochrona przed błędami i nieuprawnionym ujawnianiem danych

- Błędy połączenia z bazą lub naruszenia constraintów są przechwytywane i zamieniane na przyjazne komunikaty (bez pokazywania użytkownikowi szczegółów SQL ani struktury bazy).
- Aplikacja nie wyświetla wprost haseł ani pełnych danych wrażliwych – np. hasła przechowywane są w bazie w formie haszy, a w aplikacji nigdzie nie są prezentowane w postaci jawnej.

Współpraca z bezpieczeństwem na poziomie bazy

- Kontrola dostępu na poziomie roli w aplikacji jest spójna z rolami i uprawnieniami zdefiniowanymi w bazie (rola kierownik / rola farmaceuta).
- Aplikacja korzysta z konta technicznego posiadającego uprawnienia zgodne z rolą zalogowanego użytkownika oraz z mechanizmu `app.user_id` do logowania operacji w `LogOperacji`.

4. Implementacja systemu baz danych

W tym rozdziale opisano wdrożenie schematu bazy danych PostgreSQL dla systemu apteki, wraz z ograniczeniami integralności, mechanizmami przetwarzania danych, zabezpieczeniami (role i uprawnienia) oraz testami wykonanymi na danych przykładowych.

4.1. Tworzenie tabel i definiowanie ograniczeń

Struktura bazy danych została utworzona na podstawie modelu relacyjnego przygotowanego w pgModeler i wdrożona do serwera PostgreSQL poprzez uruchomienie skryptu DDL w narzędziu administracyjnym (pgAdmin).

W ramach wdrożenia utworzono schematy logiczne:

- apteka (dane operacyjne: leki, recepty, sprzedaże, osoby, klienci, lekarze itd.)
- magazyn (dostawcy, dostawy, partie, surowce i gospodarka magazynowa)
- użytkownicy (użytkownicy systemu, role i rejestr operacji)

Na poziomie bazy zdefiniowano ograniczenia integralności, aby wymusić poprawność danych niezależnie od aplikacji:

- PRIMARY KEY - jednoznaczna identyfikacja rekordów w tabelach.
- FOREIGN KEY - spójność referencyjna (np. recepta odnosi się do istniejącego lekarza).
- UNIQUE - unikalność atrybutów identyfikacyjnych (np. kod EAN w tabeli leków).
- CHECK - walidacja domen (np. dopuszczalne wartości typu surowca).

Ograniczenia te minimalizują ryzyko niespójności (np. duplikaty identyfikatorów, rekordy osierocone) oraz pozwalają na jednoznaczną weryfikację poprawności danych w testach.

4.2. Implementacja mechanizmów przetwarzania danych

Część logiki przetwarzania danych została zaimplementowana po stronie bazy, aby uprościć zapytania po stronie aplikacji oraz zapewnić jednolite raportowanie i audyt.

Zastosowane mechanizmy:

- Widoki (VIEW):
- Zestaw widoków raportowych agreguje i porządkuje dane (np. stan magazynowy leków oraz sprzedaż dzienna).
- Funkcje i wyzwalacze (FUNCTION + TRIGGER):
- Dla wybranych tabel skonfigurowano wyzwalacze, które rejestrują operacje INSERT/UPDATE/DELETE w tabeli logów. Log zawiera m.in. typ operacji, nazwę encji oraz klucz rekordu.

Dzięki temu aplikacja może korzystać z gotowych struktur (widoków) oraz otrzymuje automatyczny audyt zmian bez konieczności ręcznego dopisywania logiki w warstwie GUI.

Zaimplementowane widoki

19 `SELECT * FROM magazyn.v_stan_magazynu_lekow;`

	id_leku integer	nazwa character varying	ilosc_dostepna bigint	najblizsza_data_waznosc date
1	3	Amoksycylina 500 ...	145	2026-08-30
2	4	Cetirizine 10 mg	160	2026-09-29
3	1	Ibuprofen 200 mg	115	2026-07-01
4	6	Metformina 500 mg	190	2026-11-28
5	5	Omeprazol 20 mg	175	2026-10-29
6	2	Paracetamol 500 mg	130	2026-07-31

20 `SELECT * FROM magazyn.v_stan_magazynu_surowcow;`

Data Output Messages Notifications

	id_surowca integer	nazwa_surowca character varying	ilosc_dostepna bigint	najblizsza_data_waznosci date
1	1	Ethanol 96%	60	2026-03-21
2	4	Gliceryna	90	2026-05-20
3	3	Kwas salicylowy	80	2026-04-30
4	5	Mentol	100	2026-06-09
5	2	Wazelina biała	70	2026-04-10
6	6	Woda oczyszczona	110	2026-06-29

21 `SELECT * FROM apteka.v_sprzedaz_dzienna;`

Data Output Messages Notifications

	data date	obrot numeric
1	2025-12-11	13.49
2	2025-12-12	16.99
3	2025-12-13	40.98
4	2025-12-14	23.99
5	2025-12-15	27.49
6	2025-12-16	61.98

Rys. 4.1. Wyniki działania widoków raportowych w pgAdmin: stan magazynu leków, stan magazynu surowców oraz sprzedaż dzienna.

Implementacja indexów

23 `SELECT schemaname, tablename, indexname, indexdef`
 24 `FROM pg_indexes`
 25 `WHERE schemaname IN ('apтека','magazyn')`
 26 `ORDER BY schemaname, tablename, indexname;`

Data Output Messages Graph Visualiser X Notifications

Showing rows: 1 to 29 Page No: 1 of 1

	schemaname name	tablename name	indexname name	indexdef text
1	apтека	adresy	adresy_pk	CREATE UNIQUE INDEX adresy_pk ON apteka.adresy USING btree (id_adresu)
2	apтека	klienci	id_klienta	CREATE UNIQUE INDEX id_klienta ON apteka.klienci USING btree (id_klienta)
3	apтека	klienci	klienci_uq	CREATE UNIQUE INDEX klienci_uq ON apteka.klienci USING btree (id_osoby_osoby)
4	apтека	klienci	pesel	CREATE UNIQUE INDEX pesel ON apteka.klienci USING btree (pesel)
5	apтека	lekarze	lekarze_pk	CREATE UNIQUE INDEX lekarze_pk ON apteka.lekarze USING btree (id_lekarza)
6	apтека	lekarze	lekarze_uq	CREATE UNIQUE INDEX lekarze_uq ON apteka.lekarze USING btree (id_osoby_osoby)
7	apтека	lekarze	numer_PWZ	CREATE UNIQUE INDEX "numer_PWZ" ON apteka.lekarze USING btree ("numer_PWZ")

Rys. 4.2. Lista indeksów w bazie danych (podgląd w pgAdmin / katalogach systemowych).

4.3. Implementacja uprawnień i innych zabezpieczeń

Zabezpieczenia dostępu zrealizowano na poziomie bazy danych poprzez role oraz przydział uprawnień do schematów i tabel (GRANT). Przyjęto zasadę najmniejszych uprawnień - każda rola ma tylko te prawa, które są niezbędne do wykonywania zadań.

Role systemowe:

- farmaceuta - rola operacyjna (np. praca na danych bieżących, bez pełnych praw administracyjnych).
- kierownik - rola rozszerzona (zarządzanie danymi i dostęp do szerszego zakresu operacji).

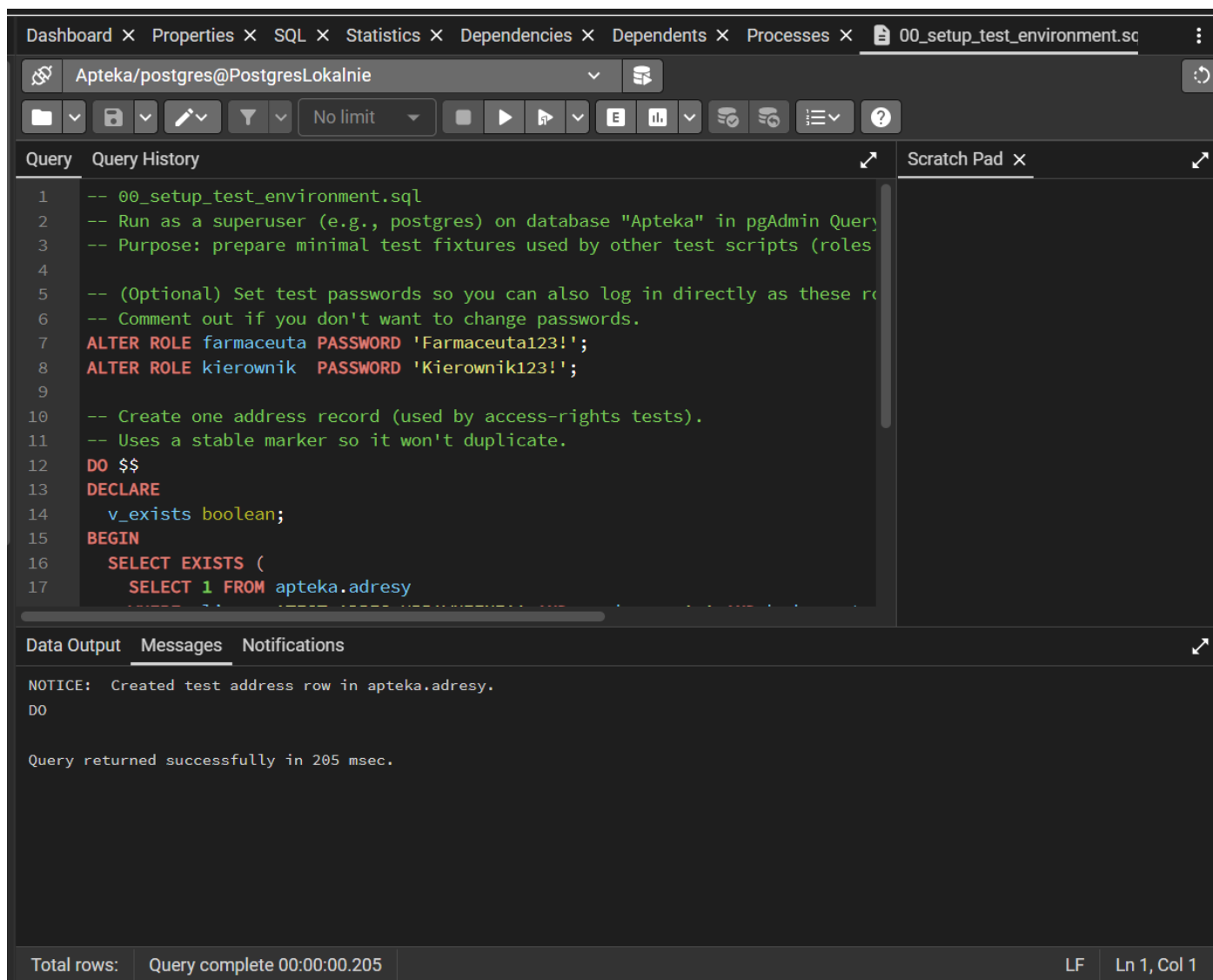
Uprawnienia nadano osobno dla schematów, tabel oraz sekwencji (wymagane do poprawnego działania kolumn autoinkrementacyjnych). W praktyce pozwoliło to wykazać, że operacje niedozwolone są blokowane przez bazę niezależnie od zachowania aplikacji.

Przykładowe założenia bezpieczeństwa testowane w projekcie:

- farmaceuta ma ograniczony dostęp do operacji usuwania (DELETE) w wybranych tabelach.
- kierownik posiada pełniejszy zakres CRUD dla tabel magazynowych i części danych operacyjnych.
- dostęp do tabel audytowych (logów operacji) jest przyznany wyłącznie roli kierownik.

4.4. Testowanie bazy danych na przykładowych danych

Testy wykonano w pgAdmin (Query Tool) na danych przykładowych generowanych automatycznie. Wyniki potwierdzano komunikatami z panelu Messages oraz zapytaniami kontrolnymi SELECT.



The screenshot shows the pgAdmin Query Tool interface. The top toolbar includes icons for file operations, query execution, and settings. The main query editor contains a SQL script for setting up a test environment. The script includes comments, role password updates, and a query to check for an existing address record. The bottom panel shows the 'Messages' tab with a success message and the execution time.

```
1  -- 00_setup_test_environment.sql
2  -- Run as a superuser (e.g., postgres) on database "Apteka" in pgAdmin Query
3  -- Purpose: prepare minimal test fixtures used by other test scripts (roles
4
5  -- (Optional) Set test passwords so you can also log in directly as these ro
6  -- Comment out if you don't want to change passwords.
7  ALTER ROLE farmaceuta PASSWORD 'Farmaceuta123!';
8  ALTER ROLE kierownik PASSWORD 'Kierownik123!';
9
10 -- Create one address record (used by access-rights tests).
11 -- Uses a stable marker so it won't duplicate.
12 DO $$
13 DECLARE
14     v_exists boolean;
15 BEGIN
16     SELECT EXISTS (
17         SELECT 1 FROM apteka.adresy
```

NOTICE: Created test address row in apteka.adresy.
DO

Query returned successfully in 205 msec.

Total rows: Query complete 00:00:00.205 LF Ln 1, Col 1

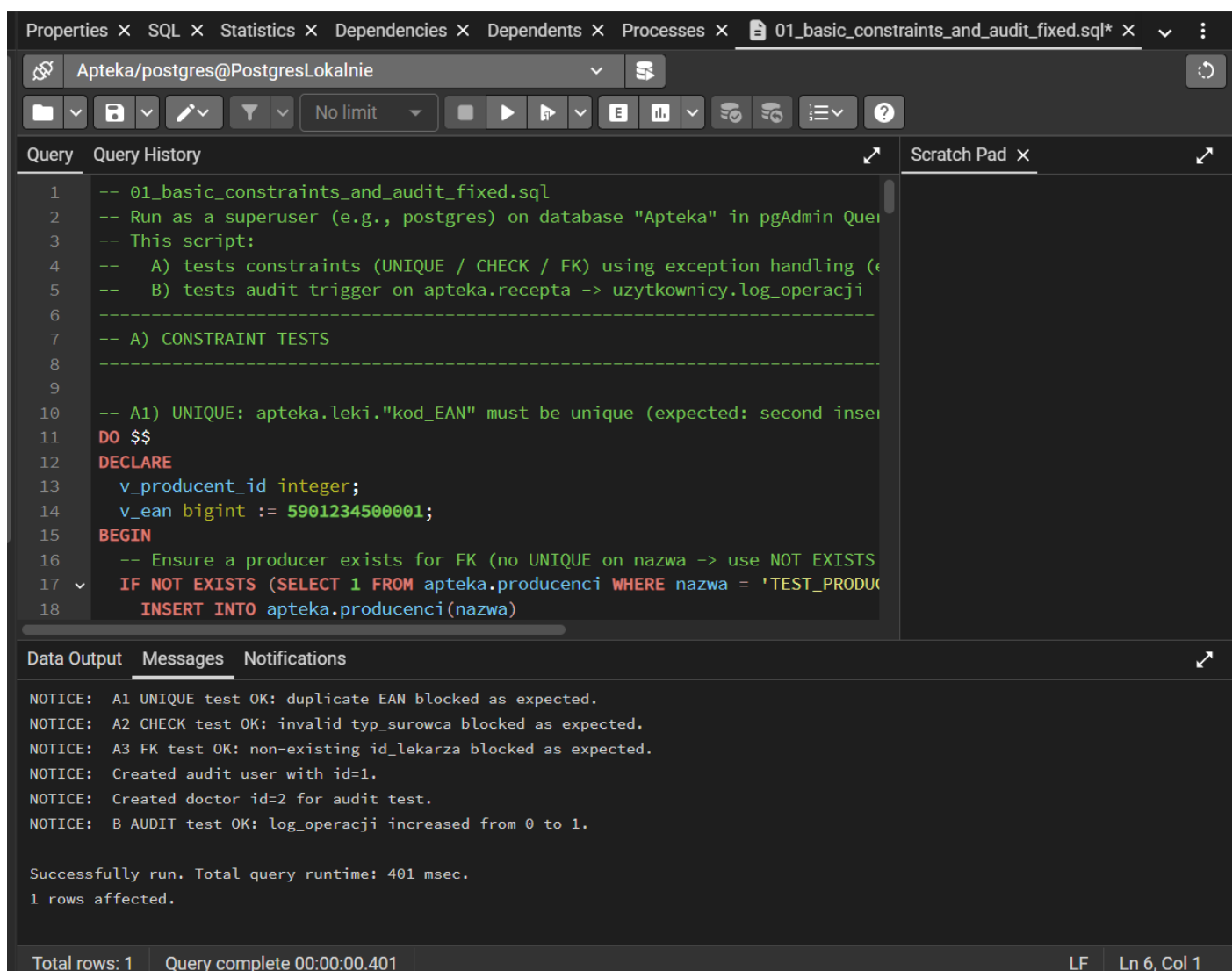
4.4.1. Testy poprawności ograniczeń (integralność danych)

Wykonano trzy podstawowe testy negatywne, które mają zakończyć się błędem na poziomie bazy:

- UNIQUE: próba wstawienia dwóch leków z tym samym kodem EAN (drugi INSERT odrzucony).
- CHECK: próba wstawienia surowca z niedozwolonym typem (odrzucone przez CHECK).
- FOREIGN KEY: próba wstawienia recepty wskazującej na nieistniejącego lekarza (odrzucone przez FK).

4.4.2. Test audytu (triggery i funkcje)

Aby potwierdzić działanie audytu, wykonano operacje INSERT/UPDATE/DELETE na tabeli recept. Następnie sprawdzano, czy w tabeli logów pojawiają się nowe wpisy opisujące operację oraz wskazujące rekord, którego dotyczyła zmiana.



The screenshot shows the pgAdmin interface with a SQL query executed. The query is a script for testing constraints and audit triggers. The output messages show that the tests passed successfully.

```
1  -- 01_basic_constraints_and_audit_fixed.sql
2  -- Run as a superuser (e.g., postgres) on database "Apteka" in pgAdmin Quer
3  -- This script:
4  --   A) tests constraints (UNIQUE / CHECK / FK) using exception handling (e
5  --   B) tests audit trigger on apteka.recepta -> uzytkownicy.log_operacji
6
7  -- A) CONSTRAINT TESTS
8  -----
9
10 -- A1) UNIQUE: apteka.leki."kod_EAN" must be unique (expected: second inser
11 DO $$
12 DECLARE
13     v_producent_id integer;
14     v_ean bigint := 5901234500001;
15 BEGIN
16     -- Ensure a producer exists for FK (no UNIQUE on nazwa -> use NOT EXISTS
17     IF NOT EXISTS (SELECT 1 FROM apteka.producenci WHERE nazwa = 'TEST_PRODU
18         INSERT INTO apteka.producenci(nazwa)
```

Output Messages:

```
NOTICE: A1 UNIQUE test OK: duplicate EAN blocked as expected.
NOTICE: A2 CHECK test OK: invalid typ_surowca blocked as expected.
NOTICE: A3 FK test OK: non-existing id_lekarza blocked as expected.
NOTICE: Created audit user with id=1.
NOTICE: Created doctor id=2 for audit test.
NOTICE: B AUDIT test OK: log_operacji increased from 0 to 1.

Successfully run. Total query runtime: 401 msec.
1 rows affected.
```

Rys. 4.3. Uruchomienie testów integralności (UNIQUE/CHECK/FK) w pgAdmin – przykład błędów blokowanych przez bazę.

4.4.3. Testy praw dostępu (role i GRANT)

Testy wykonano poprzez logowanie w pgAdmin jako użytkownicy powiązani z rolami farmaceuta oraz kierownik. Dla każdej roli uruchomiono zestaw zapytań, które powinny przejść (dozwolone operacje) oraz zestaw zapytań, które powinny zostać zablokowane (permission denied).

- farmaceuta: potwierdzenie możliwości SELECT/INSERT/UPDATE w wybranych obszarach oraz blokady DELETE tam, gdzie nie przyznano uprawnień.
- kierownik: potwierdzenie szerszych uprawnień CRUD oraz dostępu do logów operacji.
- weryfikacja komunikatów błędów w panelu Messages jako dowód poprawnego działania zabezpieczeń

The screenshot shows the pgAdmin interface with a SQL query executed. The query is as follows:

```
181 BEGIN
182 BEGIN
183 EXECUTE 'SET LOCAL ROLE kierownik';
184 EXECUTE 'DELETE FROM apteka.recepta WHERE id_recepty = 1';
185 RAISE WARNING 'K5 FAILED: kierownik unexpectedly can DELETE from apteka.recepta';
186 EXCEPTION
187 WHEN insufficient_privilege THEN
188 RAISE NOTICE 'K5 OK: kierownik cannot DELETE from apteka.recepta (as designed)';
189 END;
190 END $do$;
191
192 RESET ROLE;
193
```

The Messages panel shows the following output:

```
NOTICE: F1 OK: farmaceuta can SELECT magazyn.dostawcy (count=0).
NOTICE: F2 OK: farmaceuta cannot INSERT into magazyn.dostawcy (permission denied as expected).
NOTICE: F3 OK: farmaceuta cannot SELECT uzytkownicy.log_operacji.
NOTICE: F4 OK: farmaceuta cannot INSERT into apteka.recepta.
NOTICE: F5 OK: farmaceuta cannot DELETE from apteka.klienci.
NOTICE: K1 OK: kierownik can INSERT/UPDATE/DELETE apteka.klienci (test client id=1).
NOTICE: K2 OK: kierownik can DELETE magazyn.surowce (id=1).
NOTICE: K3 OK: kierownik can INSERT magazyn.dostawcy.
NOTICE: K4 OK: kierownik can SELECT uzytkownicy.log_operacji (count=0).
NOTICE: K5 OK: kierownik cannot DELETE from apteka.recepta (as designed).
RESET
```

Query returned successfully in 142 msec.

Total rows: Query complete 00:00:00.142 LF Ln 193, Col 1

Rys. 4.4. Testy praw dostępu (role i GRANT) w pgAdmin – operacje dozwolone i blokowane (permission denied).

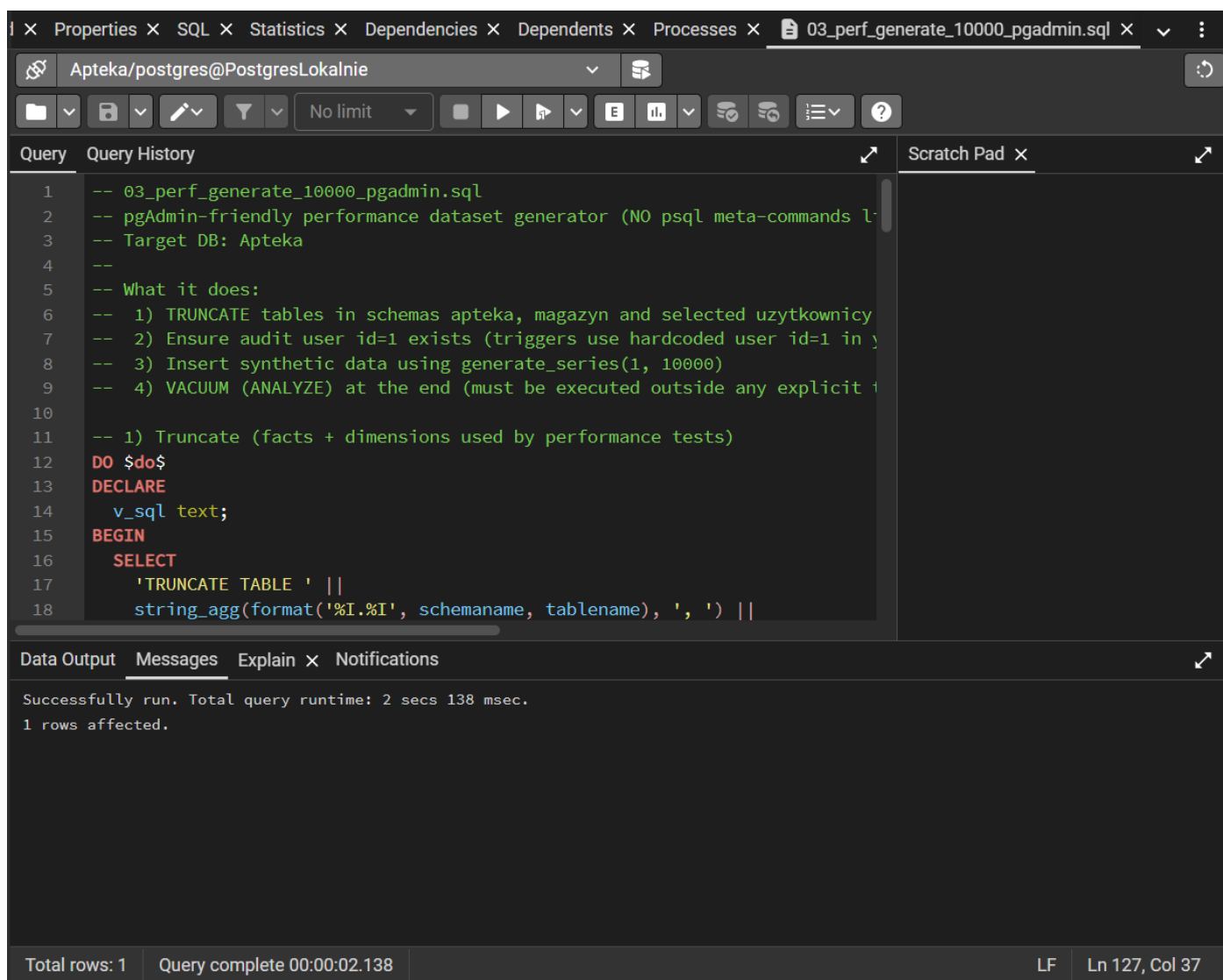
4.4.4. Testy wydajnościowe (z indeksami i bez indeksów)

Wydajność oceniono poprzez porównanie czasu wykonania zapytań dla trzech rozmiarów danych: 10 000, 20 000. Dla każdego rozmiaru wykonano pomiar w dwóch wariantach: z indeksami wydajnościowymi oraz po ich czasowym usunięciu.

Metodyka:

- Dane testowe generowano automatycznie w SQL z użyciem funkcji `generate_series(1, N)`, aby zapewnić powtarzalność oraz skalowanie wolumenu.
- Po załadowaniu danych wykonywano `VACUUM (ANALYZE)`, aby zaktualizować statystyki planera zapytań.
- Pomiary wykonywano z użyciem `EXPLAIN (ANALYZE, BUFFERS)`, zapisując plan (Index Scan vs Seq Scan) i Execution Time.
- Porównywano zapytania dobrane pod indeksy (wyszukiwanie po nazwie leku/surowca oraz filtrowanie po dacie realizacji recepty).

Tak przygotowany zestaw testów pozwala wykazać wpływ indeksów na czas wykonania zapytań oraz uzasadnić ich zastosowanie w projekcie.



The screenshot shows a PostgreSQL client interface with a dark theme. The top toolbar includes icons for file operations, query execution, and settings. The main window is titled '03_perf_generate_10000_pgadmin.sql' and displays a SQL script. The script is a PL/pgSQL function that truncates tables, inserts synthetic data using `generate_series`, and performs a `VACUUM (ANALYZE)`. The 'Data Output' tab at the bottom shows the successful execution of the script, indicating a total query runtime of 2 seconds and 138 milliseconds, with 1 row affected. The status bar at the bottom indicates 'Total rows: 1', 'Query complete 00:00:02.138', and the current cursor position at 'Ln 127, Col 37'.

```
1  -- 03_perf_generate_10000_pgadmin.sql
2  -- pgAdmin-friendly performance dataset generator (NO psql meta-commands l
3  -- Target DB: Apteka
4  --
5  -- What it does:
6  -- 1) TRUNCATE tables in schemas apteka, magazyn and selected uzytkownicy
7  -- 2) Ensure audit user id=1 exists (triggers use hardcoded user id=1 in y
8  -- 3) Insert synthetic data using generate_series(1, 10000)
9  -- 4) VACUUM (ANALYZE) at the end (must be executed outside any explicit t
10
11 -- 1) Truncate (facts + dimensions used by performance tests)
12 DO $$
13 DECLARE
14     v_sql text;
15 BEGIN
16     SELECT
17         'TRUNCATE TABLE ' ||
18         string_agg(format('%I.%I', schemaname, tablename), ', ') ||
```

Successfully run. Total query runtime: 2 secs 138 msec.
1 rows affected.

Total rows: 1 Query complete 00:00:02.138 LF Ln 127, Col 37

Rysunek 4.5 Wygenerowanie zestawu testowego 10 000 rekordów (skrypt 03_perf_generate_10000_pgadmin.sql)

Dashboard × Properties × SQL × Statistics × Dependencies × Dependents × Processes × 03_perf_generate_20000_pgadmin.sql ×

Apteka/postgres@PostgresLokalnie

Query Query History Scratch Pad ×

```
113 END,
114 'g'
115 FROM generate_series(1, 20000) AS i;
116
117 -- 9) Prescriptions (20000) -> index candidate: idx_recepta_data_realizacji
118 -- Triggers will write into uzytkownicy.log_operacji using user id=1.
119 INSERT INTO apteka.recepta(
120     data_wystawienia, data_realizacji, "data_ważności", id_sprzedazy_sprzedaze, id_klienta_k
121 )
122 SELECT
123     CURRENT_DATE - (i % 365),
124     CURRENT_DATE - (i % 60),
125     CURRENT_DATE + 30,
126     NULL,
127     1 + (i % 20000),
128     1 + (i % 200),
129     NULL
130 FROM generate_series(1, 20000) AS i;
```

Data Output Messages Explain × Notifications

INSERT 0 20000

Query returned successfully in 5 secs 850 msec.

Total rows: Query complete 00:00:05.850 LF Ln 130, Col 37

Rysunek 4.6. Wygenerowanie zestawu testowego 20 000 rekordów (skrypt 03_perf_generate_20000_pgadmin.sql)

Query Query History Scratch Pad ×

```
1 -- Performance indexes OFF (drop only the extra performance indexes)
2
3 BEGIN;
4
5 DROP INDEX IF EXISTS apteka.idx_leki_nazwa;
6 DROP INDEX IF EXISTS magazyn.idx_surowce_nazwa;
7 DROP INDEX IF EXISTS apteka.idx_recepta_data_realizacji;
8
9 COMMIT;
```

Data Output Messages Explain × Notifications

COMMIT

Query returned successfully in 176 msec.

Total rows: Query complete 00:00:00.176 LF Ln 1, Col 69

Rysunek 4.7. Wyłączenie indeksów wydajnościowych (skrypt 03_perf_indexes_off.sql) poprzez DROP INDEX IF EXISTS Dzięki temu możliwe jest wykonanie tej samej serii zapytań w wariancie **bez indeksów**. Operacja zakończona COMMIT.

The screenshot shows a PostgreSQL client window with the following components:

- Top Bar:** Contains tabs for Dashboard, Properties, SQL, Statistics, Dependencies, Dependents, Processes, and a file named 03_perf_indexes_on.sql.
- Toolbar:** Includes icons for file operations, query execution, and other database functions.
- Query Editor:** Displays the following SQL script:

```
1 BEGIN;  
2  
3 CREATE INDEX IF NOT EXISTS idx_leki_nazwa ON apteka.leki  
4 USING btree (nazwa ASC NULLS LAST)  
5 INCLUDE (nazwa);  
6  
7 CREATE INDEX IF NOT EXISTS idx_surowce_nazwa ON magazyn.surowce  
8 USING btree (nazwa_surowca ASC NULLS LAST)  
9 INCLUDE (nazwa_surowca);  
10  
11 CREATE INDEX IF NOT EXISTS idx_recepta_data_realizacji ON apteka.recepta  
12 USING btree (data_realizacji)  
13 INCLUDE (data_realizacji);  
14  
15 COMMIT;  
16
```
- Messages Panel:** Shows the following output:

```
NOTICE: relation "idx_leki_nazwa" already exists, skipping  
NOTICE: relation "idx_surowce_nazwa" already exists, skipping  
NOTICE: relation "idx_recepta_data_realizacji" already exists, skipping  
COMMIT  
  
Query returned successfully in 166 msec.
```
- Status Bar:** Displays "Total rows: Query complete 00:00:00.166" and "LF Ln 1, Col 1".

Rysunek 7. Włączenie indeksów wydajnościowych (skrypt 03_perf_indexes_on.sql) dla testów porównawczych: idx_leki_nazwa, idx_surowce_nazwa, idx_recepta_data_realizacji. Komunikaty already exists, skipping oznaczają, że indeksy były już utworzone. Operacja zakończona COMMIT.

Testy wydajnościowe z włączonymi indexami ilość danych 10 tysięcy

```
25 -- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)
26 SELECT *
27 FROM magazyn.surowce
28 WHERE nazwa_surowca = (
29     SELECT nazwa_surowca FROM magazyn.surowce ORDER BY id_surowca OFFSET 5000 LIMIT 1
30 );
31
```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 191 msec.
1 rows affected.

```
19 -- Q2) Range query on data_realizacji (good for idx_recepta_data_realizacji)
20 SELECT *
21 FROM apteka.recepta
22 WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE
23 ORDER BY data_realizacji DESC;
24
25 -- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)
26 SELECT *
27 FROM magazyn.surowce
28 WHERE nazwa_surowca = (
29     SELECT nazwa_surowca FROM magazyn.surowce ORDER BY id_surowca OFFSET 5000 LIMIT 1
30 );
31
```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 178 msec.
1 rows affected.

```
12 -- Q1) Point lookup on medicine name (good for idx_leki_nazwa)
13 SELECT *
14 FROM apteka.leki
15 WHERE nazwa = (
16     SELECT nazwa FROM apteka.leki ORDER BY id_leku OFFSET 5000 LIMIT 1
17 );
18
19 -- Q2) Range query on data_realizacji (good for idx_recepta_data_realizacji)
20 SELECT *
21 FROM apteka.recepta
22 WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE
23 ORDER BY data_realizacji DESC;
24
```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 223 msec.
1 rows affected.

Testy wydajnościowe z wyłączonymi indexami ilość danych 10 tysięcy

```

16 -- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)
17 SELECT *
18 FROM magazyn.surowce
19 WHERE nazwa_surowca = (
20     SELECT nazwa_surowca FROM magazyn.surowce ORDER BY id_surowca OFFSET 5000 LIMIT 1
21 );
22

```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 236 msec.
1 rows affected.

```

3 -- Q1) Point lookup on medicine name (good for idx_leki_nazwa)
4 SELECT *
5 FROM apteka.leki
6 WHERE nazwa = (
7     SELECT nazwa FROM apteka.leki ORDER BY id_leku OFFSET 5000 LIMIT 1
8 );
9
10 -- Q2) Range query on data_realizacji (good for idx_recepta_data_realizacji)
11 SELECT *
12 FROM apteka.recepta
13 WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE
14 ORDER BY data_realizacji DESC;
15
16 -- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)
17 SELECT *
18 FROM magazyn.surowce
19 WHERE nazwa_surowca = (

```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 146 msec.
1 rows affected.

```

10 -- Q2) Range query on data_realizacji (good for idx_recepta_data_realizacji)
11 SELECT *
12 FROM apteka.recepta
13 WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE
14 ORDER BY data_realizacji DESC;
15
16 -- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)
17 SELECT *
18 FROM magazyn.surowce
19 WHERE nazwa_surowca = (

```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 217 msec.
1 rows affected.

Testy wydajnościowe z włączonymi indexami 20 tysięcy danych

```
25 -- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)
26 SELECT *
27 FROM magazyn.surowce
28 WHERE nazwa_surowca = (
29     SELECT nazwa_surowca FROM magazyn.surowce ORDER BY id_surowca OFFSET 5000 LIMIT 1
30 );
31
```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 264 msec.
1 rows affected.

```
12 -- Q1) Point lookup on medicine name (good for idx_leki_nazwa)
13 SELECT *
14 FROM apteka.leki
15 WHERE nazwa = (
16     SELECT nazwa FROM apteka.leki ORDER BY id_leku OFFSET 5000 LIMIT 1
17 );
18
```

Data Output Messages Explain × Notifications

Successfully run. Total query runtime: 197 msec.
1 rows affected.

12/17/2025 12:23:06 PM	2671	350 msec
Date	Rows affected	Duration

Copy Copy to Query Editor

```
SELECT *
FROM apteka.recepta
WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CUR
ORDER BY data_realizacji DESC;
```

Messages

Successfully run. Total query runtime: 350 msec. 2671 rows affected.

Testy wydajnościowe z wyłączonymi indexami 20 tysięcy danych

Query

Query History

Scratch Pad x

1

-- 03_perf_run_queries_pgadmin.sql

2

3

-- Q1) Point lookup on medicine name (good for idx_leki_nazwa)

4

SELECT *

5

FROM apteka.leki

6

WHERE nazwa = (

7

SELECT nazwa FROM apteka.leki ORDER BY id_leku OFFSET 5000 LIMIT 1

8

);

9

10

-- Q2) Range query on data_realizacji (good for idx_recepta_data_realizacji)

11

SELECT *

12

FROM apteka.recepta

13

WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE

14

ORDER BY data_realizacji DESC;

15

16

-- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)

17

SELECT *

18

FROM magazyn.surowce

19

WHERE nazwa surowca = (

Data Output

Messages

Notifications

Successfully run. Total query runtime: 325 msec.
1 rows affected.

Total rows: 1

Query complete 00:00:00.325

LF

Ln 4, Col 1

Query

Query History

Scratch Pad x

1

-- 03_perf_run_queries_pgadmin.sql

2

3

-- Q1) Point lookup on medicine name (good for idx_leki_nazwa)

4

SELECT *

5

FROM apteka.leki

6

WHERE nazwa = (

7

SELECT nazwa FROM apteka.leki ORDER BY id_leku OFFSET 5000 LIMIT 1

8

);

9

10

-- Q2) Range query on data_realizacji (good for idx_recepta_data_realizacji)

11

SELECT *

12

FROM apteka.recepta

13

WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE

14

ORDER BY data_realizacji DESC;

15

16

-- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)

17

SELECT *

18

FROM magazyn.surowce

19

WHERE nazwa surowca = (

Data Output

Messages

Notifications

Successfully run. Total query runtime: 246 msec.
2671 rows affected.

Total rows: 2671

Query complete 00:00:00.246

LF

Ln 11, Col 1

The screenshot shows a PostgreSQL query editor with a dark theme. The main window displays a SQL query with line numbers 4 through 22. The query is a complex SELECT statement involving multiple tables and subqueries. It includes comments like '-- Q2) Range query on data_realizacji' and '-- Q3) Point lookup on raw material name'. The query is executed, and the results are shown in the 'Data Output' tab at the bottom. The output indicates that the query was successfully run, took 209 msec, and affected 1 row. The status bar at the bottom shows 'Total rows: 1', 'Query complete 00:00:00.209', and the current cursor position 'Ln 17, Col 1'.

```
4 SELECT *
5 FROM apteka.leki
6 WHERE nazwa = (
7     SELECT nazwa FROM apteka.leki ORDER BY id_leku OFFSET 5000 LIMIT 1
8 );
9
10 -- Q2) Range query on data_realizacji (good for idx_recepta_data_realizacji)
11 SELECT *
12 FROM apteka.recepta
13 WHERE data_realizacji BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE
14 ORDER BY data_realizacji DESC;
15
16 -- Q3) Point lookup on raw material name (good for idx_surowce_nazwa)
17 SELECT *
18 FROM magazyn.surowce
19 WHERE nazwa_surowca = (
20     SELECT nazwa_surowca FROM magazyn.surowce ORDER BY id_surowca OFFSET 5000 LIMIT 1
21 );
22
```

Data Output Messages Notifications

Successfully run. Total query runtime: 209 msec.
1 rows affected.

Total rows: 1 Query complete 00:00:00.209 LF Ln 17, Col 1

5. Implementacja i testy aplikacji

System został zaimplementowany jako **aplikacja desktopowa** w języku **C#** z interfejsem użytkownika wykonanym w **Avalonia UI**. Aplikacja komunikuje się bezpośrednio z relacyjną bazą danych **PostgreSQL**, a model bazy został przygotowany w **pgModeler 1.2.2**. Architektura rozwiązania opiera się na rozdzieleniu warstw: **GUI → logika aplikacji → warstwa dostępu do danych (repozytoria)**, co ułatwia utrzymanie kodu oraz ogranicza powiązanie widoków z detalami SQL.

W bazie danych zastosowano:

- podział na logiczne schematy (obszary domeny),
- ograniczenia integralności (PK/FK/UNIQUE/CHECK),
- widoki raportowe (do odczytu i raportowania),
- indeksy (optymalizacja wyszukiwania i raportów),
- mechanizm audytu (logowanie operacji INSERT/UPDATE/DELETE),
- role i uprawnienia (separacja dostępu dla farmaceuty i kierownika).

W aplikacji zastosowano:

- logowanie użytkownika i rozpoznawanie roli,
- parametryzowane zapytania SQL,
- transakcje dla operacji wieloetapowych,
- walidację danych w formularzach (m.in. PESEL, pola wymagane adresu).

5.1. Instalacja i konfigurowanie systemu

5.1.1. Wymagania środowiskowe

Wymagane elementy:

- **PostgreSQL** (serwer bazy danych).
- **Sterownik ODBC do PostgreSQL** (np. PostgreSQL Unicode).
- Środowisko uruchomieniowe **.NET** dla aplikacji (zgodnie z ustawieniami projektu).
- System operacyjny: Windows lub Linux.

Narzędzia pomocnicze:

- **pgAdmin** (do uruchamiania skryptów, podglądu schematów, widoków, indeksów, uprawnień).
- **pgModeler 1.2.2** (modelowanie i dokumentowanie bazy danych).

5.1.2. Instalacja bazy danych (PostgreSQL)

Krok 1 — utworzenie bazy

W pgAdmin tworzymy bazę danych (np. Apteka). W części przypadków skrypt z pgModeler zawiera CREATE DATABASE, ale praktycznie często wykonuje się to osobno (żeby uniknąć problemów z wykonywaniem całego pliku naraz).

Krok 2 — uruchomienie skryptu DDL

W pgAdmin otwieramy Query Tool i uruchamiamy skrypt tworzący:

- schematy (np. apteka, magazyn, uzytkownicy),
- tabele z kluczami głównymi i obcymi,
- ograniczenia (UNIQUE, CHECK),
- widoki,
- funkcje i triggerzy audytu,
- role oraz nadania uprawnień (GRANT).

Krok 3 — dane testowe (seed/inserty)

Dane testowe można wprowadzić osobnym skryptem INSERT.

Typowy błąd seedowania, o którym rozmawialiśmy:

duplicate key value violates unique constraint ... Key (id_...) already exists

Najczęstsze przyczyny i rozwiązania:

1. **Wstawianie ID „na sztywno”** (np. id_osoby = 1) mimo że rekord już istnieje
→ rozwiązać przez użycie DEFAULT/sekwencji albo ustalić nowe ID.
2. **Sekwencja „nie nadąża” za ręcznymi insertami**
→ po seedzie zsynchronizować sekwencję, np. (schemat i nazwa sekwencji zależne od tabeli):

```
SELECT setval('apтека.osoby_id_osoby_seq', (SELECT MAX(id_osoby) FROM apteka.osoby));
```

To zapobiega sytuacji, że nextval() próbuje dać ID już istniejące.

5.1.3. Konfiguracja aplikacji (połączenie ODBC)

Aplikacja łączy się z bazą danych przez **ODBC**. Konfiguracja połączenia obejmuje:

- driver (PostgreSQL Unicode),
- host (np. localhost),
- port (domyślnie 5432),
- database (np. Apteka),
- użytkownik/hasło.

W warstwie aplikacji za połączenie odpowiada DatabaseService, który:

- tworzy OdbcConnection,
- otwiera połączenie,
- (opcjonalnie) ustawia parametr sesji app.user_id, wykorzystywany w audycie w bazie.

Ważne dla audytu:

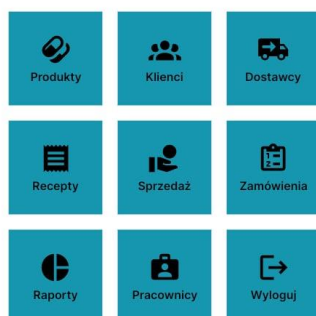
Aplikacja może poprawnie ustawiać app.user_id, ale w bazie funkcje triggerów muszą rzeczywiście odczytywać current_setting('app.user_id'). W wersji, którą omawialiśmy, w triggerach pojawia się podejście z wartością stałą

(v_id_uzytkownika := 1) — to warto doprowadzić do wariantu docelowego, żeby log był przypisany do realnie zalogowanego użytkownika.

5.2. Instrukcja użytkowania aplikacji

5.2.1. Ekran główny (nawigacja po modułach)

Po zalogowaniu użytkownik widzi ekran główny z kafelkami modułów: m.in. Produkty/Leki, Klienci, Dostawcy, Recepty, Sprzedaż, Zamówienia, Raporty, Pracownicy oraz Wyloguj. Kliknięcie kafelka otwiera odpowiedni widok funkcjonalny. Dostępność części modułów może zależeć od roli (np. „Pracownicy” typowo tylko dla kierownika).



Rys. 5.1. Ekran główny aplikacji – menu kafelkowe modułów.

5.2.2. Moduł „Klienci” – przeglądanie danych

Widok „Klienci” składa się z:

- pola wyszukiwania (Szukaj) i przycisków filtrowania/czyszczenia,
- tabeli klientów z podstawowymi danymi (ID, imię i nazwisko, PESEL, telefony),
- panelu szczegółów (formularz) umożliwiającego podgląd i edycję,
- przycisków operacji: Dodaj / Edytuj / Usuń oraz Zapisz / Anuluj.

← Wstecz

Klienci

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300

Dodaj

Edytuj

Usuń

ID:

Imię:

Nazwisko:

PESEL:

Numer:

Adres:

Zapisz

Anuluj

Rys. 5.2. Widok „Klienci” – lista klientów oraz panel szczegółów.

5.2.3. Dodawanie i edycja klienta – walidacja PESEL

Podczas edycji klienta aplikacja weryfikuje poprawność numeru PESEL. W przypadku wprowadzenia niepoprawnego numeru pole zostaje wyróżnione (np. czerwone obramowanie), a pod polem wyświetla się komunikat błędu. Funkcja ta zapobiega zapisywaniu danych niezgodnych z wymaganiami.

← Wstecz

Klienci

Szukaj:

wyszukaj

Filtruj

Wyczyść

1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300
0			Brak numeru

Dodaj

Edytuj

Usuń

ID:

0

Imię:

Nazwisko:

PESEL:

Niepoprawny numer PESEL

Numer:

Brak numeru

Adres:

Zapisz

Anuluj

Rys. 5.3. Walidacja PESEL – komunikat „Niepoprawny numer PESEL”.

5.2.4. Edycja adresów – okno modalne z walidacją pól

Adresy są edytowane w osobnym oknie modalnym „Adresy”. Formularz wymaga uzupełnienia pól obowiązkowych (np. kraj, numer domu, miejscowość, kod pocztowy). Brak danych powoduje wyświetlenie komunikatów walidacyjnych przy konkretnych polach.

← Wstecz

Klienci

Szukaj:

wyszukaj

Wyczyść

ID

Imię i Nazwisko

PESEL

Telefony

1

Piotr Wiśniewski

90010112345

123456789, Służbowy: 600200200

2

Maria Zielińska

85050554321

500200300

0

Adresy

ID: 0

Kraj:

Pole "Kraj" nie może być puste

Ulica:

Numer Domu:

Pole "NumerDomu" nie może być puste

Numer Lokal:

Miejscowość:

Pole "Miejscowosc" nie może być puste

Kod Pocztowy:

Pole "KodPocztowy" nie może być puste

ZapiszAnuluj

Zamknij

ID: 0

Imię:

Nazwisko:

PESEL:

Niepoprawny numer PESEL

Numer: Brak numeru

Adres:

ZapiszAnuluj

Dodaj

Edytuj

Usuń

Rys. 5.4. Okno „Adresy” – walidacja pól wymaganych (puste pola).

5.2.5. Edycja numerów telefonu – osobny widok/moduł

Numerы telefonu klienta obsługiwane są w osobnym widoku, który zawiera:

- listę numerów (z opisem, np. „Służbowy”),
- pola do dodawania i edycji numeru,
- przyciski: Dodaj / Edytuj / Usuń oraz Zapisz / Anuluj / Zamknij.

← Wstecz

Klienci

Szukaj:

wyszukaj

Wyczyść

ID

Imię i Nazwisko

PESEL

Telefony

1

Piotr Wiśniewski

90010112345

123456789, Służbowy: 600200200

2

Maria Zielińska

85050554321

500200300

0

Brak numeru

Szukaj:

wyszukaj

Wyczyść

ID

Opis

Numer

Dodaj

Edytuj

Usuń

ID:

Opis:

Numer:

ZapiszAnuluj

Zamknij

ID:

Imię:

Nazwisko:

PESEL:

Niepoprawny numer PESEL

Numer:

Brak numeru

Adres:

ZapiszAnuluj

Rys. 5.5. Okno numerów telefonu – widok listy (pusty / tryb dodawania).

← Wstecz

Klienci

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Opis	Numer
5		123456789
6	Służbowy	600200200

Dodaj

Edytuj

Usuń

ID:

Opis:

Numer:

Zapisz

Anuluj

Zamknij

Imię:

Piotr

Nazwisko:

Wiśniewski

PESEL:

90010112345

Numer:

123456789, Służbowy: 600200200

Adres:

Polska, Długa, 10/2, Warszawa, 00-001

Zapisz

Anuluj

Rys. 5.6. Okno numerów telefonu – lista z wpisami (np. „Służbowy”).

5.2.6. Podgląd danych klienta (dane powiązane)

Po wybraniu klienta z tabeli, w panelu szczegółów widoczne są dane podstawowe (imię, nazwisko, PESEL), lista numerów telefonów oraz adres. Użytkownik może przejść w tryb edycji i zapisać zmiany.

← Wstecz

Klienci

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300
0			Brak numeru

Dodaj

Edytuj

Usuń

ID:

1

Imię:

Piotr

Nazwisko:

Wiśniewski

PESEL:

90010112345

Numer:

123456789, Służbowy: 600200200

Adres:

Polska, Długa, 10/2, Warszawa, 00-001

Zapisz

Anuluj

Rys. 5.7. Panel szczegółów klienta – PESEL, telefony, adres.

5.2.7. Wyszukiwanie klientów

Pole „Szukaj” umożliwia szybkie filtrowanie listy klientów po fragmencie danych (np. imię/nazwisko). Po wpisaniu frazy lista zawęży się do pasujących rekordów. Przycisk „Wyczyść” przywraca pełny widok.

← Wstecz

Klienci

Szukaj:

Piot

Filtruj

Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200

Dodaj

Edytuj

Usuń

ID:

Imię:

Nazwisko:

PESEL:

Numerzy:

Adres:

Zapisz

Anuluj

Rys. 5.8. Wyszukiwanie klientów – przykład filtrowania po frazie „Piot”.

5.2.8. Moduł „Leki” – lista produktów oraz szczegóły i warianty

Widok „Leki” prezentuje listę produktów wraz z informacjami o producencie, substancji czynnej oraz oznaczeniem „bez recepty”. W panelu szczegółów prezentowane są warianty (np. EAN, dawka, ilość/dostępność, postać). Ułatwia to kontrolę asortymentu i dostępności.

← Wstecz

Leki

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Nazwa	Producent	Substancja Czynna	Bez Recepty
1	Apap	Polpharma S.A.	Paracetamol	☒
2	Ibuprom	Polpharma S.A.	Ibuprofen	☒
3	Duomox	Bayer Sp. z o.o.	Amoxicillinum	☐

Dodaj

Edytuj

Usuń

ID:

1

EAN:

590000000001

Dawka:

500 mg

Ilość:

12

Dostępność:

109

Postać:

Kapsułka

Nazwa:

Apap

Producent:

Polpharma S.A.

Adres:

Polska, Fabryczna, 100/A, Łódź, 90-000

Bez Recepty:

☒

Substancja Czynna:

Paracetamol

Zapisz

Anuluj

Rys. 5.9. Widok „Leki” – lista produktów i szczegóły (EAN/dawka/dostępność).

5.3. Testowanie opracowanych funkcji systemu

Testy wykonano w dwóch obszarach:

1.

testy aplikacji (manualne) – zgodność z przypadkami użycia i poprawność działania GUI,
2.

testy bazodanowe – integralność danych, uprawnienia, audyt oraz (opcjonalnie) wydajność.

5.3.1. Testy funkcjonalne aplikacji (manualne)

T1 — Poprawne logowanie

- Kroki: podać poprawny login i hasło → klik „Zaloguj”.
- Oczekiwane: przejście do menu głównego, aktywna rola użytkownika, gotowość modułów.

Logowanie

Nazwa użytkownika:

Hasło:

T2 — Błędne logowanie

- Kroki: błędny login/hasło.
- Oczekiwane: brak przejścia do menu, komunikat o błędzie/odrzućcie logowania.

Logowanie

Nazwa użytkownika:

Hasło:

Zaloguj

Błędny login lub hasło

T3 — Dostęp zależny od roli

- Kroki: zalogować się jako farmaceuta i kierownik.
- Oczekiwane: różnice w dostępnych modułach (np. moduł pracowników ograniczony).

Od:

1	styczeń	0001
---	---------	------

Do: 16 styczeń 2032

Sprzedaż Magazyn

Type	Nazwa	Ilość	Data ważności
	Apap 500 mg x 12	109	2026-04-25
	Duomox 1000 mg x 20	19	2027-01-01
	Ibuprom 200 mg x 10	49	2026-06-30
	Mentol	1000	2026-12-31
	Wazelina biała	5000	2028-01-01

Od:

1	styczeń	0001
---	---------	------

Do:	31	grudzień	2026
-----	----	----------	------

[Sprzedaż](#) [Magazyn](#)

Typ	Nazwa	Ilość	Data ważności
	Apap 500 mg x 12	109	2026-04-25
	Ibuprom 200 mg x 10	49	2026-06-30
	Mentol	1000	2026-12-31

Do:	16	styczeń	2026
-----	----	---------	------

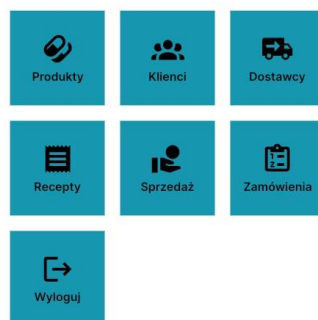
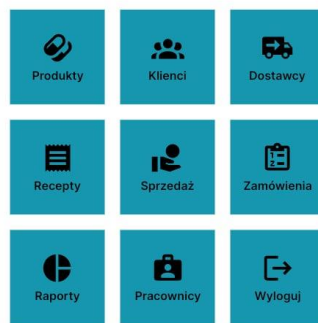
Magazyn

ID	Data	Kwota	
7	2026-01-16 00:00	0,00 zł	
13	2026-01-16 00:00	0,00 zł	
14	2026-01-16 00:00	2,30 zł	
15	2026-01-16 00:00	2,30 zł	
3	2025-01-04 00:00	50,00 zł	
2	2025-01-03 00:00	45,00 zł	
1	2025-01-03 00:00	25,50 zł	

Do:	15	styczeń	2026
-----	----	---------	------

Magazyn

ID	Data	Kwota	
3	2025-01-04 00:00	50,00 zł	
1	2025-01-03 00:00	25,50 zł	
2	2025-01-03 00:00	45,00 zł	



T4 — Walidacja PESEL

- Kroki: edycja klienta → wpisanie błędnego PESEL → próba zapisu.
- Oczekiwane: komunikat „Niepoprawny numer PESEL”, brak zapisu.

← Wstecz

Klienci

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300
0	Adam Konieczny	68123000276	Brak numeru

Dodaj

Edytuj

Usuń

ID:

2

Imię:

Maria

Nazwisko:

Zielińska

PESEL:

85050554321

Numer:

500200300

Adres:

Polska, Apteczna, 5/2137, Wrocław, 50-100

Zapisz

Anuluj

← Wstecz

Klienci

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300
0	Adam Konieczny	68123000276	Brak numeru

Dodaj

Edytuj

Usuń

ID:

0

Imię:

Adam

Nazwisko:

Konieczny

PESEL:

68123000276

Niepoprawny numer PESEL

Numer:

Brak numeru

Adres:

Niemcy, 12, Görlitz, 02826

Zapisz

Anuluj

T5 — Walidacja danych adresowych

- Kroki: utworzyć okno „Adresy” → zostawić pola obowiązkowe puste → próba zapisu.
- Oczekiwane: komunikaty walidacyjne przy polach, brak zapisu.

Adresy

ID: 0

Kraj: Niemcy

Ulica:

Numer Domu: 12

Numer Lokalu:

Miejscowość: Görlitz

Kod Pocztowy:

Pole "KodPocztowy" nie może być puste

Zapisz

Anuluj

Zamknij

← Wstecz **Klienci**

Szukaj: Filtruj Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300
0	Adam Konieczny	68123000276	Brak numeru

Adresy

ID: 0

Kraj: Niemcy

Ulica:

Numer Domu: 12

Numer Lokalu:

Miejscowość: Görlitz

Kod Pocztowy: 02826

Zapisz Anuluj

Zamknij

Dodaj Edytuj Usuń

ID: 0

Imię: Adam

Nazwisko: Konieczny

PESEL: 68123000276

Numer: Brak numeru

Adres:

Zapisz Anuluj

Niepoprawny numer PESEL

T6 — Zarządzanie numerami telefonu

- Kroki: otworzyć okno numerów → dodać numer z opisem → zapisać → sprawdzić widok klienta.
- Oczekiwane: numer pojawia się w liście, jest powiązany z klientem.

← Wstecz

Klienci

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Imię i Nazwisko	PESEL	Telefony
1	Piotr Wiśniewski	90010112345	123456789, Służbowy: 600200200
2	Maria Zielińska	85050554321	500200300

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Opis	Numer
5	Prywatny	123456789
6	Służbowy	600200200

Dodaj

Edytuj

Usuń

ID:

Opis:

Numer:

Zapisz

Anuluj

Zamknij

Imię:

Piotr

Nazwisko:

Wiśniewski

PESEL:

90010112345

Numery:

123456789, Służbowy: 600200200

Adres:

Polska, Długa, 10/2, Warszawa, 00-001

Zapisz

Anuluj

T7 — Moduł „Leki”

- Kroki: wejść w leki → wybrać rekord → sprawdzić warianty i dostępność.
- Oczekiwane: prawidłowe dane wariantów (EAN, dawka), zgodna dostępność.

Szukaj:

Filtruj

Wyczyść

ID	Nazwa	Producent ↑	Substancja Czynna	Bez Recepty
3	Duomox	Bayer Sp. z o.o.	Amoxicillinum	☐
1	Apap	US Pharmacia Sp. z o.o.	Paracetamol	☒
2	Ibuprom	US Pharmacia Sp. z o.o.	Ibuprofen	☒

DodajEdytujUsuń

ID:1

EAN

Dawka

Ilość

Dostępność

Postać

Nazwa:Apap

5909990296033

500 mg

12

109

Tabletka

Producent:US Pharmacia Sp. z o.o.

5909990296040

500 mg

24

0

Tabletka

Adres:Polska, Fabryczna, 100, Wrocław, 50-507

Bez Recepty:☐

Substancja Czynna:Paracetamol

ZapiszAnuluj

Szukaj:

Filtruj

Wyczyść

ID	Nazwa	Producent ↑	Substancja Czynna	Bez Recepty
4	Paracetamol Biofarm	Biofarm Sp. z o. o.	Paracetamol	☒
1	Apap	US Pharmacia Sp. z o.o.	Paracetamol	☒

DodajEdytujUsuń

ID:

Nazwa:

Producent:

Adres:

Bez Recepty:☐

Substancja Czynna:

EAN

Dawka

Ilość

Dostępność

Postać

ZapiszAnuluj

← Wstecz

Leki

Szukaj:

wyszukaj

Filtruj

Wyczyść

ID	Nazwa	Producent ↑	Substancja Czynna	Bez Recepty
3	Duomox	Bayer Sp. z o.o.	Amoxicillinum	☐
1	Apap	US Pharmacia Sp. z o.o.	Paracetamol	☒
2	Ibuprom	US Pharmacia Sp. z o.o.	Ibuprofen	☒

ID:

2

EAN

5907377130475

Dawka

200 mg

Ilość

20

Dostępność

49

Postać

Tabletka

Nazwa:

Ibuprom

Producent:

US Pharmacia Sp. z o.o.

Adres:

Polska, Fabryczna, 100, Wrocław, 50-507

Bez Recepty:

☐

Substancja Czynna:

Ibuprofen

Dodaj

Edytuj

Usuń

Zapisz

Anuluj

5.4. Omówienie wybranych rozwiązań programistycznych

5.4.1. Implementacja interfejsu dostępu do bazy danych

Warstwa dostępu do danych została zorganizowana w postaci:

- klasy serwisowej odpowiedzialnej za połączenie (DatabaseService),
- repozytoriów realizujących operacje na konkretnych obszarach (np. logowanie, klienci, adresy, telefony, leki, raporty).

Cechy implementacji:

- **ODBC** jako mechanizm połączenia z PostgreSQL,
- **parametryzacja zapytań** (zamiast konkatencji stringów),
- **transakcje** w operacjach wieloetapowych (np. usuwanie powiązań i czyszczenie rekordów nieużywanych),
- możliwość ustawienia parametru sesji app.user_id (pod audyt).

Korzyści:

- większe bezpieczeństwo,
- spójność obsługi błędów,
- czytelny podział odpowiedzialności.

5.4.2. Implementacja wybranych funkcjonalności systemu

Moduł Klienci

- lista klientów z możliwością filtrowania,
- panel szczegółów z danymi klienta,
- obsługa danych powiązanych:
 - adresy edytowane w modalu,
 - telefony edytowane w osobnym oknie listy.

Moduł Leki/Produkty

- lista leków z informacjami o producencie i substancji czynnej,
- oznaczenie „bez recepty”,
- panel wariantów (EAN, dawka, dostępność, postać).

Raporty

- wykorzystanie widoków raportowych po stronie bazy i prosty odczyt w aplikacji,
- ograniczenie skomplikowanych agregacji w kodzie aplikacji (ciężar przeniesiony na bazę).

5.4.3. Implementacja mechanizmów bezpieczeństwa

1) Role i uprawnienia w bazie

Zastosowano role (np. farmaceuta i kierownik) oraz GRANT-y na schematach/tabelach/sekwencjach. Dzięki temu ograniczono dostęp użytkowników do operacji zgodnie z zakresem obowiązków.

2) Audyt operacji w bazie

Triggery logują zmiany danych do tabeli logów. W wariancie docelowym identyfikator użytkownika jest pobierany z parametru sesji (app.user_id) ustawianego przez aplikację po zalogowaniu.

3) Bezpieczeństwo w aplikacji

- logowanie użytkownika i rozpoznawanie roli,
- walidacja danych (PESEL, pola wymagane),
- parametryzowane zapytania do bazy.

Rekomendacje ulepszeń (docelowo)

- wzmocnienie przechowywania haseł (solenie i/lub algorytm typowo do haseł),
- usunięcie ewentualnego debug-logowania danych wrażliwych,
- dopięcie audytu tak, by zawsze rejestrował realnego użytkownika.

6. Podsumowanie i wnioski

- Zrealizowany system spełnia założone cele projektu. Powstał **kompletny system zarządzania apteką**, obejmujący relacyjną bazę danych w PostgreSQL oraz aplikację desktopową umożliwiającą obsługę kluczowych procesów biznesowych, takich jak zarządzanie klientami, lekami, magazynem, sprzedażą oraz raportowaniem.
- Baza danych została zaprojektowana w sposób uporządkowany i czytelny poprzez **podział na schematy domenowe** (apteka, magazyn, użytkownicy), co ułatwia zarówno utrzymanie, jak i dalszy rozwój systemu. Spójność i poprawność danych zapewniają **mechanizmy integralności referencyjnej**, takie jak klucze główne i obce oraz ograniczenia UNIQUE i CHECK.
- Warstwa aplikacji została oddzielona od bazy danych poprzez zastosowanie **repozytoriów i ustandaryzowanego połączenia ODBC**, co zwiększa elastyczność rozwiązania i pozwala na łatwiejszą modyfikację logiki biznesowej bez ingerencji w strukturę danych.
- Dodatkowo system został wyposażony w **mechanizmy bezpieczeństwa**, takie jak role bazodanowe, kontrola dostępu do danych oraz audyt operacji zapisu, co jest szczególnie istotne w kontekście danych wrażliwych przetwarzanych w aptece.
-

6.1 Najważniejsze osiągnięcia projektu

- zaprojektowanie **spójnego i znormalizowanego modelu danych w PostgreSQL** z logicznym podziałem na obszary domenowe,
- zastosowanie **ograniczeń integralności (PK, FK, UNIQUE, CHECK)** zapewniających poprawność danych na poziomie bazy,
- implementacja **widoków raportowych** oraz **indeksów**, wspierających raportowanie i wydajne wyszukiwanie danych,
- wdrożenie **mechanizmu audytu zmian danych** z wykorzystaniem funkcji i triggerów,
- rozdzielenie uprawnień poprzez **role bazodanowe** (kierownik, farmaceuta) i zasadę minimalnych uprawnień,
- stworzenie **funkcjonalnej aplikacji desktopowej** z czytelnym GUI, walidacją danych oraz obsługą danych powiązanych (adresy, telefony),
- zapewnienie **spójnej współpracy aplikacji z bazą danych** poprzez warstwę repozytoriów i ODBC.
-

6.2 Wyzwania, z którymi się mierzyliśmy

- **Złożoność domeny apteki**
Konieczność odwzorowania rzeczywistych procesów (recepty, sprzedaż, partie magazynowe, receptura) wymagała przemyślanego modelu relacji i tabel pośrednich.
- **Projekt integralności danych**
Wyzwanie stanowiło dobranie odpowiednich ograniczeń tak, aby baza była odporna na błędne dane, a jednocześnie nie ograniczała poprawnych scenariuszy biznesowych.
- **Bezpieczeństwo i uprawnienia**
Precyzyjne rozdzielenie uprawnień pomiędzy role oraz ograniczenie bezpośredniego dostępu do wrażliwych danych wymagało starannego zaprojektowania ról i GRANT-ów.
- **Spójność audytu z aplikacją**
Powiązanie operacji wykonywanych w aplikacji z użytkownikiem w bazie danych (parametr sesji) było istotnym i nietrywialnym elementem integracji.
- **Czytelność i ergonomia GUI**
Interfejs musiał być jednocześnie prosty dla użytkownika końcowego i wystarczająco elastyczny, aby obsługiwać różne moduły oraz dane powiązane.